AVIGILON™

Programming Notes

Access Control Manager™ REST API

© 2015, 2019 - 2023, Avigilon Corporation. All rights reserved. AVIGILON, the AVIGILON logos, AVIGILON ACCESS CONTROL CENTER, ACC, and ACCESS CONTROL MANAGER are trademarks of Avigilon Corporation. Digital Monitoring Products® is a registered trademark of Digital Monitoring Products. Remote Link is a trademark of Digital Monitoring Products, Incorporated. HID, HID GLOBAL, HID Mercury, HID Mobile Access, APERIO and VERTX are trademarks or registered trademarks of HID Global, ASSA ABLOY AB, or its affiliate(s) in the US and other countries. Allegion, ENGAGE technology and Schlage are trademarks of Allegion plc, its subsidiaries and/or affiliates in the United States and other countries. Other names or logos mentioned herein may be the trademarks of their respective owners. The absence of the symbols ™ and ® in proximity to each trademark in this document or at all is not a disclaimer of ownership of the related trademark.

This document has been compiled and published using product descriptions and specifications available at the time of publication. The contents of this document and the specifications of the products discussed herein are subject to change without notice. Avigilon Corporation reserves the right to make any such changes without notice. Neither Avigilon Corporation nor any of its affiliated companies: (1) guarantees the completeness or accuracy of the information contained in this document; or (2) is responsible for your use of, or reliance on, the information. Avigilon Corporation shall not be responsible for any losses or damages (including consequential damages) caused by reliance on the information presented herein.

Avigilon Corporation
avigilon.com

PDF-ACM-RESTAPI

Revision: 13 - EN

20230607

Revisions

ACM Release	Description s
June 2023	Other updates: • *Supported Web Endpoints and API Versions* on page 34 • *Show Panel Status* on page 148 • *Show Subpanel Status* on page 164 • *Create Appliance Backup* on page 51
Release 6.42, March 2023	Note added: • *Secure login and password information* on page 13
Release 6.36, October 2022	REST API version 1.12: Intrusion panel requests • *Supported Web Endpoints and API Versions* on page 34 • *Intrusion Panels* on page 141
Release 6.32, April 2022	REST API version 1.11: `installed=<true|false>` parameter for `/rest` endpoints • *Supported Web Endpoints and API Versions* on page 34 • *Get Installed or Uninstalled Doors* on page 77 • *Find Installed or Uninstalled Panels* on page 147 • *Find Installed or Uninstalled Subpanels* on page 164 • *Get Installed or Uninstalled Roles* on page 157 `installed=<true|false>` and `inuse=<true|false>` parameters for `/rest` endpoints • *Get Installed or Uninstalled Inputs* on page 140 REST API version 1.10: • Enhanced endpoints are no longer 'Preview'

ACM Release	Description s
Release 6.30, March 2022	(Preview) Enhanced endpoints for ACM appliance, delegations, doors, identities, inputs, panels, roles, and subpanels: • *Supported Web Endpoints and API Versions* on page 34 • *Get Delegations* on page 79 • *Get System Status* on page 53 Canned macro request: • *Supported Web Endpoints and API Versions* on page 34 • *Canned Macros* on page 58
January 2022	Collaboration request and output examples: • *Supported Web Endpoints and API Versions* on page 34 • *Collaborations* on page 62
September 2021	Identity code samples: • *Update Roles* on page 123 • *Update* on page 124 • User Defined Fields: *Update* on page 126, *Get Update* on page 127
Release 6.22, July 2021	ACC™ identity login transactions and partition parameter in input response: • *Supported Web Endpoints and API Versions* on page 34 • *ACC™ Software Identity Logins* on page 38 • *Inputs* on page 136 Note about the Force Password Change field: • *Identities* on page 104
Release 6.20, May 2021	User Defined List request and output: • *Supported Web Endpoints and API Versions* on page 34 • *User Defined Lists* on page 175
Release 6.18, April 2021	Subpanel output update: • *Supported Web Endpoints and API Versions* on page 34 • *Subpanels* on page 160
Release 6.14, December 2020	Door mode fix in door status output: • *Supported Web Endpoints and API Versions* on page 34 • *Door Status* on page 64

ACM Release	Description s
Release 6.8, August 2020	Support for getting Schlage® and Von Duprin® linked devices that operate in IP/PoE (Power over Ethernet) mode and managed through the Schalge ENGAGE™ Gateway, and getting Mercury and HID™ subpanels: • *Sending a Panel Change for Linked Devices Request* on page 22 • *Supported Web Endpoints and API Versions* on page 34 • *Panel Change* on page 69
Release 6.2, November 2019	Support for search patterns in advanced searches: • *Identities* on page 104 Support for creating and updating tokens for Personal Identification Verification (PIV) and PIV-Interoperable (PIV-I) cards: • *Token Create* on page 168 • *Token Update* on page 171

Table of Contents

Revisions 3

Introduction 13

Best Practices and Recommendations 13

Use the UI or collaboration first 13

Secure login and password information 13

Keep it simple 14

Check return codes and data 14

Don't delete it 14

Test, test, test 14

Programming 15

Sample REST Calls and Responses 17

Sending a Door List Request 17

Sending a Panel Change for Linked Devices Request 22

Generic Code Samples 25

Find All 25

Find One 26

Create 28

Update 29

Delete 31

Door Commands 33

Supported Web Endpoints and API Versions 34

ACC™ Software Identity Logins 38

Model 38

Output 38

Access Groups 39

Model: access_groups 39

Find All 39

Output 40

Create 42

Update 44

Alarms 46

Acknowledge Alarm 46

Clear Alarm 48

Appliance 49
Model: gateways 49
Find All 50
Create Appliance Backup 51
Model 51
Output 52
Request Example 52
XML 52
Get System Status 53
Request Examples 53
XML 53
JSON 53
Areas 54
Model: areas 54
Find All 54
Create 54
Output 56
Card Formats 57
Model: card_formats 57
Find All 57
Canned Macros 58
Model 58
Find All 58
Request Examples 59
Find Canned Macro by DN 59
Find Canned Macro by Door 59
Find Canned Macro by Input 60
Response Examples 60
Find All 60
Find Canned Macro by DN 61
Collaborations 62
Model 62
Output 62
Initiate Collaboration 62
Doors 64

Model: doors 64
Door Commands 68
Panel Change 69
panel 69
linked-device 69
Output Examples 70
Door Elevator Grant 71
Find All 72
Door Status 73
Response Examples 75
Get Installed or Uninstalled Doors 77
Request and Response Examples 77
XML 77
JSON 78
Delegations 79
Model 79
Get Delegations 79
Request Examples 79
XML 79
JSON 79
Event Types 80
Model: evtypes 80
Find All 80
Update 81
Events 84
Model: points 84
Find All 85
Global Actions 86
Model: ioactions 86
Find All 86
Create 88
Update 90
Execute 92
Groups 94
Model: groups 94

Find All 94
Create 94
Update 96
Holidays 99
Model: holidays 99
Find All 99
Create 99
Output 101
Update 101
Identities 104
Model: identities 104
Find All 106
Response Examples 107
Find One 112
Response Example 114
Advanced Search 116
Find by External System ID 121
Find by Multiple Criteria 121
Find by Token Number 121
Advanced Filter Searches 121
Search by Modify Date 122
Find Identities by Role 122
Create 122
Update Roles 123
Update 124
User Defined Fields 125
Update 126
Get Update 127
Photo 128
Create 128
Read 130
Update 132
Delete 133
Inputs 136
Model: inputs 136
Example Responses 137

All Inputs with Partitioning 137
All Inputs for a Single Subpanel 138
Get Installed or Uninstalled Inputs 140
Request Examples 140
XML 140
JSON 140
Intrusion Panels 141
Model 141
Output 142
List Intrusion Panels 142
Request Examples 142
XML 142
JSON 142
Update Intrusion Panel 142
Request Example 142
XML 142
Delete Intrusion Panel 143
Request Example 143
XML 143
Map Templates 144
Model: map_tmpls 144
Outputs 145
Model: outputs 145
Panels 146
Model: panels 146
Find Installed or Uninstalled Panels 147
Request Examples 148
XML 148
JSON 148
Show Panel Status 148
Request Example 148
XML 148
Response Example 148
Photos 152
Readers 153

Model: readers 153

Roles 155

Model: roles 155

Find All Roles 156

Request and Response Examples 156

XML 156

Get Installed or Uninstalled Roles 157

Request Examples 158

XML 158

JSON 158

Schedules 159

Model: timezones 159

Subpanels 160

Model: subps 160

Sample Responses 161

All Subpanels for a Single Panel 161

All Subpanels for All Panels 163

Find Installed or Uninstalled Subpanels 164

Request Examples 164

XML 164

JSON 164

Show Subpanel Status 164

Request Example 164

XML 164

Response Example 164

Tokens 165

Model: tokens 165

Find All 166

Find by Identity 167

Token Create 168

Token Delete 170

Token Update 171

User Commands 173

Model: ucmds 173

User Defined Tabs 174

Model: udef_tabs 174

User Defined Lists 175

Model: udef_vals 175

Output 175

Get All Values in a List 175

User Defined Fields 177

Model: userdefs 177

Introduction

The Access Control Manager™ (ACM) software includes a powerful application programming interface (API) for programmers of custom integrations to the ACM) software.)Note: Licensing fees apply. The ACM API extends even further the power of the collaborations in the ACM software (Access Control Manager Collaboration Platform™ Software).

The ACM software capabilities are exposed to programmers through an easy-to-use REST Web Service. All web-service requests sent to the ACM API are formatted in standard HTTP over SSL. The data sent to and received from the ACM API are formatted in standard XML. All commands sent to the ACM system must go through authentication and authorization similar to the ACM web interface.

Best Practices and Recommendations

The following best practices and recommendations apply.

Note: Verify the ACM API version of the ACM system you are integrating with. For more information, see *Supported Web Endpoints and API Versions* on page 34.

Use the UI or collaboration first

Whenever possible, the ACM system user interface (UI) or collaboration is highly recommended as the first method of accessing ACM data. Consider these options before writing a custom integration.

Secure login and password information

You need to create an identity record in the ACM system with login and password credentials for your REST program. This allows you to set up transaction logging and authorization of only your REST program. You can choose to configure the ACM user interface to not log REST logins, which reduces unnecessary transactions in a polling type of REST program. You can also set the Items/Page field to display a large number of items on a page when logged in. For more information, see the My Account settings of each identity record in the ACM application.

Note: ACM Multi-Factor Authentication prevents authentication of identities through the REST API.

Important: All ACM software communications are encrypted over SSL. Authentication login and password information must also be secured appropriately in your custom REST program. At a minimum, ensure that source code permissions do not expose private information. Better yet,

implement a design that encrypt logins and passwords using public and private key encryption.

Keep it simple

To ensure that custom developed applications remain compatible with future ACM software upgrades and enhancements, keep the interfaces as simple as possible.

Check return codes and data

Always be sure to check both the HTTP return codes and the body of the message to ensure that a command was executed as expected.

Don’t delete it

Even though you can perform delete transactions from external programs, updating the installed flag to “N” in the ACM application is recommended rather than delete database entries.

Test, test, test

Like any development effort, all custom programming must be tested extensively to work as expected in all conditions. The code should also handle all error cases.

Programming

The actions exposed via the ACM API are accessed via standard HTTP(s) commands. This REST interface is very easy to use, if you can send an HTTP command you can use the ACM REST API. Like all access to the ACM application the API REST requests must use SSL, must have a login authenticated and be authorized to perform the command. All commands to the ACM REST API use standard XML requests and responses.

To determine the REST actions that you would like to call it is very helpful to first determine how to perform the same action through the ACM system UI. When navigating throughout the ACM UI take note of the URL's that are used to perform the actions you wish to emulate via the REST api. These URL's will be basically the same ones that your programs will call. Once you determine the URL's that you would like to call refer to this document for details on the values and techniques that are required for the respective calls.

The ACM system stores configuration data in an LDAP database. The unique identifier used throughout the application is known as the "Distinguished Name" (DN). In the examples below where you see :dn you need to substitute the appropriate DN value to indicate which specific record you are looking for.

Note: Technically, in most cases you can use the LDAP Common Name or 'CN' attribute however it should never be a problem to use the more specific 'DN' attribute.

In debugging your REST programs it is a help to have a look at the rails_log.txt. This file can be viewed through the UI by going to Appliance, Logs tab and clicking on the rails_log.txt link in the log file listing. Scroll to the bottom of the file and you will see the server response to the latest request. If you have experiencing trouble you can often see what the server doesn't like about your request. Be sure to check that the login and password is correct and that the model names are spelled correctly and that you have the '.xml' extension on your URLs.

Programming summary and best practices:

- Use SSL.
- Set the Content-Type = 'application/xml'.
- Create a unique identity with login and password to be used to authenticate your REST calls.
- Be sure to secure login/password information in your program.
- Send login/password information via standard http 'basic access authorization'.
- Use `HTTP-GET` for read, `HTTP-PUT` for update, `HTTP-POST` for create, `HTTP-DELETE` for delete.
- Send requests in XML format in the body of the HTTP request.
- Check the HTTP response for success or failure of the command.
- Where data is returned it will be in XML or JSON format. The format depends on the specific command issued.
- Use the Distinguished Name as the unique key where required.
- Pay attention when manipulating date fields. Date fields often require special processing and must

match the format defined for the logged in user as users can have different formatting depending on their locale.

Sample REST Calls and Responses

ACM is a browser-based application which is built leveraging REST. In general anything that you can do through the UI you can also do via REST. Type the following web service URIs into your browser and replace <my_ACM_Server> with the IP address or host name of your ACM server.

Sending a Door List Request

The ACM REST API supports the following request and response transactions to get a list of all doors.

1. Send this request:
   https://<my_ACM_Server>/doors.xml
2. You will be prompted for a login and password. Enter a valid ACM system username and password.
3. The ACM system replies with an XML-formatted list of doors.

```
<?xml version="1.0" encoding="UTF-8"?>

<doors type="array">

    <door>

        <dn>cn=1,ou=doors,cn=e3a755988b1e46bb,ou=gateway
        s,dc=plasec</dn>

        <cns type="array">

            <cn>1</cn>

    </cns>

    <objectClasses type="array">

        <objectClass>plasecDoor</objectClass>

        <objectClass>plasecDoorPanel</objectClass>

    </objectClasses>

    <plasecAddress>1</plasecAddress>

    <plasecAltname>what</plasecAltname>

    <plasecCameraDNs type="array">

<plasecCameraDN>cn=0,ou=cameras,cn=1,ou=externalsystems,cn=e3a755988
b1e46bb,ou=gateways,dc=plasec</plase

<plasecCameraDN>cn=9,ou=cameras,cn=1,ou=externalsystems,cn=e3a755988
b1e46bb,ou=gateways,dc=plasec</plase

    </plasecCameraDNs>
```

```
<plasecInstalled>TRUE</plasecInstalled>

<plasecLocation>is up here?</plasecLocation>

<plasecName>1501-1 demo door</plasecName>

<plasecVersion>0</plasecVersion>

<plasecdoorAccesscfg>0</plasecdoorAccesscfg>

<plasecdoorAccesstime>5</plasecdoorAccesstime>

<plasecdoorAccesstimemax>10</plasecdoorAccesstimemax>

<plasecdoorAgroupDNs type="array">

    <plasecdoorAgroupDN>cn=1,ou=agroups,cn=e3a755988
    b1e46bb,ou=gateways,dc=plasec</plasecdoorAgroupD
    N>

    <plasecdoorAgroupDN>cn=2,ou=agroups,cn=e3a755988
    b1e46bb,ou=gateways,dc=plasec</plasecdoorAgroupD
    N>

</plasecdoorAgroupDNs>

<plasecdoorAltaccesstime>10</plasecdoorAltaccesstime>

<plasecdoorAlthotime>60</plasecdoorAlthotime>

<plasecdoorDoNotLogRextransactions>FALSE</plasecdoorDoNotL
ogRextransactions>

<plasecdoorDoormode>8</plasecdoorDoormode>

<plasecdoorDposDN>cn=100,cn=0,cn=1,ou=panels,cn=e3a755988b1e46bb,ou=
gateways,dc=plasec</plasecdoorDposDN>

<plasecdoorFormats>1</plasecdoorFormats>

<plasecdoorHotime>30</plasecdoorHotime>

<plasecdoorLeddrvmode>1</plasecdoorLeddrvmode>

<plasecdoorMaskforced>FALSE</plasecdoorMaskforced>

<plasecdoorMaskheld>FALSE</plasecdoorMaskheld>

<plasecdoorMercaddress>0</plasecdoorMercaddress>

<plasecdoorPanel
DN>cn=1,ou=panels,cn=e3a755988b1e46bb,ou=gateways,dc=plase
c</plasecdoorPanelDN>

<plasecdoorPinattempts>0</plasecdoorPinattempts>

<plasecdoorPinlockout>0</plasecdoorPinlockout>
```

```
        <plasecdoorPintimeout>0</plasecdoorPintimeout>

        <plasecdoorPrealarm>5</plasecdoorPrealarm>

        <plasecdoorRdr1DN>cn=300,cn=0,cn=1,ou=panels,cn=e3a755988b
        1e46bb,ou=gateways,dc=plasec</plasecdoorRdr1DN>

        <plasecdoorRdrflagdenyduress>FALSE</plasecdoorRdrflagdenyd
        uress>

        <plasecdoorRdrflagdetailedevts>TRUE</plasecdoorRdrflagdeta
        iledevts>

        <plasecdoorRdrflagdoorforcedfilter>FALSE</plasecdoorRdrfla
        gdoorforcedfilter>

        <plasecdoorRdrflagenableciphermode>FALSE</plasecdoorRdrfla
        genableciphermode>

        <plasecdoorRdrflaglogallaccessused>TRUE</plasecdoorRdrflag
        logallaccessused>

        <plasecdoorRdrflagloggrantsrightaway>TRUE</plasecdoorRdrfl
        agloggrantsrightaway>

        <plasecdoorRdrflagnopulsedrstrikeonRex>FALSE</plasecdoorRd
        rflagnopulsedrstrikeonRex>

        <plasecdoorRdrflagrequiretwocardctrl>FALSE</plasecdoorRdrf
        lagrequiretwocardctrl>

        <plasecdoorRdrflagshuntrelay>FALSE</plasecdoorRdrflagshunt
        relay>

        <plasecdoorRex0DN>cn=101,cn=0,cn=1,ou=panels,cn=e3a755988b
        1e46bb,ou=gateways,dc=plasec</plasecdoorRex0DN>

<plasecdoorStrikeDN>cn=200,cn=0,cn=1,ou=panels,cn=e3a755988b1e46bb,o
u=gateways,dc=plasec</plasecdoorStrikeDN>

        <plasecdoorStrikemode>1</plasecdoorStrikemode>

        <plasecgroupDNs type="array">

            <plasecgroupDN>cn=4f6d4c48e6d54b81,ou=groups,dc=
            plasec</plasecgroupDN>

        </plasecgroupDNs>

        <plasecidentityCameraType>1</plasecidentityCameraType>

        <plasecpanelManufs type="array">

            <plasecpanelManuf>cn=1,ou=manufs,dc=plasec</plas
            ecpanelManuf>
```

```
     </plasecpanelManufs>

     </door>

     <door>

          <dn>cn=4,ou=doors,cn=e3a755988b1e46bb,ou=gateway
          s,dc=plasec</dn>

          <cns type="array">

               <cn>4</cn>

           </cns>

     <objectClasses type="array">

     <objectClass>plasecDoor</objectClass>

     <objectClass>plasecDoorPanel </objectClass>

</objectClasses>

<plasecAddress>4</plasecAddress>

<plasecInstalled>TRUE</plasecInstalled>

<plasecName>Vertx-1-1</plasecName>

<plasecdoorAccesscfg>0</plasecdoorAccesscfg>

<plasecdoorAccesstime>0</plasecdoorAccesstime>

<plasecdoorAccesstimemax>10</plasecdoorAccesstimemax>

<plasecdoorAgroupDNs type="array">

     <plasecdoorAgroupDN>cn=3,ou=agroups,cn=e3a755988b1e46bb,ou
     =gateways,dc=plasec</plasecdoorAgroupDN>

     </plasecdoorAgroupDNs>

     plasecdoorAlthotime>60</plasecdoorAlthotime>

     <plasecdoorDoNotLogRextransactions>FALSE</plasecdoorDoNotL
     ogRextransactions>

     <plasecdoorDoormode>8</plasecdoorDoormode>

<plasecdoorDposDN>cn=100,cn=1,cn=4,ou=panels,cn=e3a755988b1e46bb,ou=
gateways,dc=plasec</plasecdoorDposD

     <plasecdoorFormats>15</plasecdoorFormats>

     <plasecdoorHotime>30</plasecdoorHotime>

     <plasecdoorLeddrvmode>1</plasecdoorLeddrvmode>

     <plasecdoorLocknum>1</plasecdoorLocknum>
```

```
        <plasecdoorMaskforced>FALSE</plasecdoorMaskforced>

        <plasecdoorMaskheld>FALSE</plasecdoorMaskheld>

        <plasecdoorMercaddress>0</plasecdoorMercaddress>

        <plasecdoorOfflinedoormode>4</plasecdoorOfflinedoormode>

        <plasecdoorPanel
        DN>cn=4,ou=panels,cn=e3a755988b1e46bb,ou=gateways,dc=plase
        c</plasecdoorPanel DN>

        <plasecdoorPinattempts>0</plasecdoorPinattempts>

        <plasecdoorPinlockout>0</plasecdoorPinlockout>

        <plasecdoorPintimeout>0</plasecdoorPintimeout>

        <plasecdoorPrealarm>5</plasecdoorPrealarm>

        <plasecdoorRdr1DN>cn=300,cn=1,cn=4,ou=panels,cn=e3a755988b
        1e46bb,ou=gateways,dc=plasec</plasecdoorRdr1DN>

        <plasecdoorRdrflagdenyduress>FALSE</plasecdoorRdrflagdenyd
        uress>

        <plasecdoorRdrflagdetailedevts>TRUE</plasecdoorRdrflagdeta
        iledevts>

        <plasecdoorRdrflagenableciphermode>FALSE</plasecdoorRdrfla
        genableciphermode>

        <plasecdoorRdrflagnopulsedrstrikeonRex>FALSE</plasecdoorRd
        rflagnopulsedrstrikeonRex>

        <plasecdoorRex0DN>cn=101,cn=1,cn=4,ou=panels,cn=e3a755988b
        1e46bb,ou=gateways,dc=plasec</plasecdoorRex0DN>

    <plasecdoorStrikeDN>cn=200,cn=1,cn=4,ou=panels,cn=e3a755988b1e46bb,o
    u=gateways,dc=plasec</plasecdoorStrikeDN>

        <plasecdoorStrikemode>1</plasecdoorStrikemode>

        <plasecdoorSubpDN>cn=1,cn=4,ou=panels,cn=e3a755988b1e46bb,
        ou=gateways,dc=plasec</plasecdoorSubpDN>

        <plasecdoorUseTracking>1</plasecdoorUseTracking>

        <plasecpanelManufs type="array">

            <plasecpanelManuf>cn=2,ou=manufs,dc=plasec</plas
            ecpanelManuf>

        </plasecpanelManufs>

    </door>

    </doors>
```

What we just did was basically a send a REST call asking the ACM system to give us a list of all doors. The

ACM server received the request and sent us back a list of all doors in XML format. Using a browser to test REST calls can be an effective technique. Note that the browser will only work for GET type calls. There are various tools available to help test REST Uniform Resource Identifiers (URIs), one such tool is the Postman Chrome app.

Sending a Panel Change for Linked Devices Request

The ACM REST API supports the following request and response transactions to create a door panel in the ACM system for any linked devices managed through an external gateway.

1. Send this request:

   https://<my_ACM_Server>/doors/panel_change.xml?gatewayid=<gatewayId>&manuf=<manuf_dn>&panel=<panel_dn>

Field	Format	Definition
my_ACM_Server	String	The IP address or host name of your ACM server.
gatewayId	String	The identifier of the external gateway.
manuf	String	The identifier of the linked device manufacturer. 1 — Mercury door 2 — HID™ door 8 — Avigilon door 10 — Schlage IP and Von Duprin® devices operating in IP (PoE or 410-IP) mode
panel	String	The panel distinguished name (panel_dn) identifier.

2. When prompted for a login and password, enter a valid ACM system username and password.
3. The ACM system responds with XML-formatted fields for the external gateway modeled as a panel and any linked devices modeled as doors. For more information, see *Panel Change* on page 69.

Note: The following output is supported in ACM API version 1.03 or later.

```
<?xml version="1.0" encoding="UTF-8"?>
<panel-info>
<panel>
    <dn>cn=4,ou=panels,cn=ada363e86d3e42f1,ou=gateways,dc=plasec</d
n>
    <cns type="array">
      <cn>4</cn>
    </cns>
<entryUUID>595521b0-1f8a-103a-8b2d-9f72a4245069</entryUUID>
<ipNetworkNumber>nn.nn.nnn.nnn</ipNetworkNumber>
```

```
<objectClasses type="array">
<objectClass>plasecPanel</objectClass>
</objectClasses>
<plasecAddress>4</plasecAddress>
<plasecInstalled>FALSE</plasecInstalled>
<plasecLocalTimeStr>Canada/Pacific</plasecLocalTimeStr>
<plasecName>gwe-36DF</plasecName>
<plasecPartitionDNs type="array">
<plasecPartitionDN>cn=6cfdbc67bf6e4018,ou=partitions,dc=plasec</plas
ecPartitionDN>
</plasecPartitionDNs>
<plasecVersion>1</plasecVersion>
<plasecpanelCommtyp>4</plasecpanelCommtyp>
<plasecpanelCredential>Basic abcdef123456</plasecpanelCredential>
<plasecpanelGmzoffset>-28800</plasecpanelGmzoffset>
<plasecpanelManufs type="array">
<plasecpanelManuf>cn=10,ou=manufs,dc=plasec</plasecpanelManuf>
</plasecpanelManufs>
<plasecpanelModel>1</plasecpanelModel>
<plasecpanelOfflineto>15000</plasecpanelOfflineto>
<plasecpanelRequireTLS>TRUE</plasecpanelRequireTLS>
<plasecpanelSchlageSite>cn=ada2afc0ff7b4ae2,ou=sites,cn=10,ou=manufs
,dc=plasec</plasecpanelSchlageSite>
</panel>
<door-subps nil="true"/>
<linked-devices type="array">
<linked-device>
<dn>cn=a100000000002327,ou=linkedDevices,cn=4,ou=panels,cn=ada363e86
d3e42f1,ou=gateways,dc=plasec</dn>
<cns type="array">
<cn>a100000000002327</cn>
</cns>
<entryUUID>32a5d9f4-1f96-103a-8b39-9f72a4245069</entryUUID>
<macAddresses type="array">
<macAddress>nn:nn:nn:nn:nn:nn</macAddress>
</macAddresses>
<objectClasses type="array">
<objectClass>plasecLinkedDevice</objectClass>
</objectClasses>
<plasecLinkedDeviceLinkId>dev00000</plasecLinkedDeviceLinkId>
<plasecLinkedDeviceModelType>nde</plasecLinkedDeviceModelType>
<plasecLinkedDeviceTokenCount>0</plasecLinkedDeviceTokenCount>
<plasecName>NDE-1234</plasecName>
```

```
</linked-device>
</linked-devices>
</panel-info>
```

Generic Code Samples

The following code samples show an overview of the ACM REST API. The samples are written in the Ruby programming language but can easily be translated to your favorite language. These samples provided an overview of the basics used throughout the ACM REST API. Some calls may require extra code or special techniques to get the desired effect. Refer to the specific sample section for details on any specific supported call.

Find All

This example shows the basic technique used to get a list of all the records for a given model. In most cases you can choose the model you wish to get the data for from the Models section of this document, and substitute the appropriate URL in place of "/doors.xml" in the sample below. This is a HTTP GET request that returns an xml array of all the records for the model.

Some standards you will see throughout these examples:

- Always use https.
- Unless you have installed a custom certificate on your ACM server you will want to ensure your code does not try to verify the default self-signed certificate. We do this below with OpenSSL::SSL:VERIFY_NONE.
- Use content_type application/xml.
- Ensure you pass '.xml' on the URL (or JSON if the call uses JSON).

```
# get_doors.rb

# sample REST call to get a list of doors

#

# arguments: ServerAddress, ServerPort, Username, password

#

my_server = ARGV[0]

my_port = ARGV[1]

my_user = ARGV[2]

my_pwd = ARGV[3]


require 'net/https'

http = Net::HTTP.new(my_server, my_port)

http.use_ssl = true

http.verify_mode = OpenSSL::SSL::VERIFY_NONE
```

```
http.start do |http|

      # this request is for a list of all doors
      req = Net::HTTP::Get.new('/doors.xml')

      # gotta give our login and password for authorization and
      authentication
      req.basic_auth(my_user, my_pwd)

      # set the content type to xml for REST commands
      req["Content-Type"] = "application/xml"

      #send the REST request
      resp = http.request(req)

      puts "RESPONSE: #{resp.header} \n\n"
      puts "DATA: "
      puts "#{resp.body}"

end
```

Find One

To get an XML response with one record send a HTTP GET request with the CN or DN depending on the specific call of the record you wish to retrieve. Also note that the sample identity find one below shows a special capability of this call to optionally retrieve User Defined Fields associated with the identity record.

```
      # identity_find_one.rb
      # sample REST call to get a single Identity record
      # <twl> Avigilon, Inc.
      #
      # arguments: ServerAddress, ServerPort, Username, password
      #
      my_server = ARGV[0]
```

```
my_port = ARGV[1]

my_user = ARGV[2]

my_pwd = ARGV[3]

my_id_cn = ARGV[4]

require 'net/https'

http = Net::HTTP.new(my_server, my_port)

http.use_ssl = true

http.verify_mode = OpenSSL::SSL::VERIFY_NONE


http.start do |http|


    # The identiy common name (CN) for the desired identity
    must be passed as part of the url

    # you get the cn from the identities query. It's <CN> or
    also is part of the full Distinguished Name (DN)

    # <dn>cn=8a5692efcaf14b1d,ou=identities,dc=plasec</dn>

    # <cn>8a5692efcaf14b1d</cn>

    #

    # if we want user defined fields then add the include_
    udf=true parameter (NOTE: UDF's are a feature new in 5.6)

    my_url = "/identities/" + my_id_cn + ".xml?include_
    udf=true"

    req = Net::HTTP::Get.new(my_url)


    # gotta give our login and password for authorization and
    authentication

    req.basic_auth(my_user, my_pwd)


    # set the content type to xml for REST commands

    req["Content-Type"] = "application/xml"


    #send the REST request

    resp = http.request(req)
```

```
    puts "RESPONSE: #{resp.header} \n\n"
    puts "DATA: "
    puts "#{resp.body}"

end
```

Create

To create a new record in the ACM system send a HTTP POST request with the values to be created passed in the body of the request. The sample below shows creating a new Access Group that includes one door.

```
# access_group_create.rb
# sample REST call to create an Access Group
# <twl> Avigilon, Inc.
#
# arguments: ServerAddress, ServerPort, Username, password, gateway_CN
#
my_server = ARGV[0]
my_port = ARGV[1]
my_user = ARGV[2]
my_pwd = ARGV[3]
my_gw = ARGV[4]

require 'net/https'

http = Net::HTTP.new(my_server, my_port)
http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE

http.start do |http|

    my_url = "/access_groups.xml?gatewayid=" + my_gw
```

```
req = Net::HTTP::Post.new(my_url)


# The body of the request contains xml of the attribute(s)
to update

xml_string =

"<access_group>

     <plasecInstalled>TRUE</plasecInstalled>

     <plasecName>AccessGroup TEST</plasecName>

     <plasecTimezoneDN>cn=2,ou=timezones,dc=plasec</p
     lasecTimezoneDN>

     <plasecagroupDoorDN>cn=1,ou=doors,cn=#{my_
     gw},ou=gateways,dc=plasec</plasecagroupDoorDN>

     </access_group>"

req.body = xml_string


# gotta give our login and password for authorization and
authentication

req.basic_auth(my_user, my_pwd)


# set the content type to xml for REST commands

req["Content-Type"] = "application/xml"


#send the REST request

resp = http.request(req)


puts "RESPONSE: #{resp.header} \n\n"

puts "DATA: "

puts "#{resp.body}"


end
```

Update

In the example below we show how a HTTP PUT is used to update an existing record. In general the URL will

contain the model and the CN or DN of the record to update. The body will contain the attributes with values to be updated. Below we are updating instructions to the Door Held Event Type. These instructions will then show on the ACM Alarm Monitor screen when this type of event occurs.

```
# evtype_update.rb
# sample REST call to update an Event Type record
# <twl> Avigilon, Inc.
#
# arguments: ServerAddress, ServerPort, Username, password, evtype_cn
#
my_server = ARGV[0]
my_port = ARGV[1]
my_user = ARGV[2]
my_pwd = ARGV[3]
my_evtype_cn = ARGV[4]

require 'net/https'

http = Net::HTTP.new(my_server, my_port)
http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE

http.start do |http|

# this request is to update an Event Type
my_url = '/evtypes/' + my_evtype_cn + '.xml'
req = Net::HTTP::Put.new(my_url)

# The body of the request contains xml of the attribute(s) to update
# in this case we're adding instructions to the Event Type
xml_string = '<evtype>
<plasecetypInstructions>Guard: Physically go and check to see why
```

```
this door is being held open. Enter notes to indicate the time of
the check and what was found.</plasecetypInstructions>

</evtype>'


req.body = xml_string


# gotta give our login and password for authorization and
authentication

req.basic_auth(my_user, my_pwd)

# set the content type to xml for REST commands

req["Content-Type"] = "application/xml"


#send the REST request

resp = http.request(req)


puts "RESPONSE: #{resp.header} \n\n"

puts "DATA: "

puts "#{resp.body}"


end
```

Delete

We show the delete example, but as stated above, the recommended best practice is to use the installed flag to uninstall an object, or use the status field on Identities or Tokens to mark them as inactive instead of deleting the record from the database.

In the example below we show how a HTTP DELETE is used to remove an existing record. In general the URL will contain the model and the CN or DN of the record to delete. Be really sure you want to do this!

```
# holiday_delete.rb

# sample REST call to delete a Holiday record

# <twl> Avigilon, Inc.

#

# arguments: ServerAddress, ServerPort, Username, password, holiday_CN

#
```

```
my_server = ARGV[0]
my_port = ARGV[1]
my_user = ARGV[2]
my_pwd = ARGV[3]
my_cn = ARGV[4]

require 'net/https'

http = Net::HTTP.new(my_server, my_port)
http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE

http.start do |http|

     # this request is to delete a holiday record
     my_url = '/holidays/' + my_cn + '.xml'
     req = Net::HTTP::Delete.new(my_url)

     # gotta give our login and password for authorization and
     authentication
     req.basic_auth(my_user, my_pwd)
     # set the content type to xml for REST commands
     req["Content-Type"] = "application/xml"

     #send the REST request
     resp = http.request(req)

     puts "RESPONSE: #{resp.header} \n\n"
     puts "DATA: "
     puts "#{resp.body}"

end
```

Door Commands

To execute a command upon a door send a HTTP POST command with the DN of the door that you wish to have the command execute against. See the door commands section for the entire list of available door commands.

```
require 'net/https'

# server=YOUR SERVER, port=443 *443 is default ssl port

http = Net::HTTP.new('192.168.1.1', 443)

http.use_ssl = true


http.start do |http|

     # this request is to grant a temporary access to a door
     req =

Net::HTTP::Post.new
('/doors/cn=4,ou=doors,cn=3af5cd86cd0f4095,ou=gateways,dc=pl
asec/grant.xml')


     # give our login and password for authorization and
     authentication

     # !!IMPORTANT: Secure this in the real world!!! req.basic_
     auth('user', 'password')


     # set the content type to xml for REST commands req
     ["Content-Type"] = "application/xml"


     #send the REST request

     resp, data = http.request(req)


     # this sample program displays the response and data puts
     "RESPONSE: #{resp} \n\n"

     puts "DATA: " puts

     "#{data}"

end
```

Supported Web Endpoints and API Versions

The following table lists the ACM REST API web endpoints supported by the ACM API versions.

> **Note:** The ACM REST API web endpoints that begin with `/rest` provide enhanced search and system status reporting capabilities. API version numbering may not be applicable to all endpoints.

ACM REST API Web Endpoint	API Version
ACC Software Identity Logins	
`/me.xml`	1.07 or later
ACM Appliance	
`/gateways.xml`	1.0 or later
`/gateways/gw_backups.xml`	-
`/rest/get_sys_update.xml` `/rest/get_sys_update.json`	1.10 or later
Canned Macros	
`/canned_macros.xml`	1.0 or later
`/canned_macros.xml/<canned_macro_dn_for_input|output|door>`	1.09 or later
Collaborations	
`/identity_collabs.xml`	1.0 or later
Delegations	

`/rest/right_groups.xml`
`/rest/right_groups.json`
`/rest/right_groups.xml?attributes=<attributes_list>`
`/rest/right_groups.json?attributes=<attributes_list>`
	1.10 or later

Doors	
`/doors.xml`	1.0 or later
`/doors/panel_change.xml`	1.03 or later
`/doors/show_status.xml` `/doors/show_status.json`	1.04 or later

ACM REST API Web Endpoint	API Version
/rest/doors.xml /rest/doors.json /rest/doors.xml?attributes=<attributes_list> /rest/doors.json?attributes=<attributes_list>	1.10 or later
/rest/doors.xml?installed=<true|false> /rest/doors.json?installed=<true|false> /rest/doors.xml?attributes=<attributes_list>&installed=<true|false> /rest/doors.json?attributes=<attributes_list>?installed=<true|false>	1.11 or later
Identities	
/identities.xml	1.0 or later
/identities/code_photo.xml	1.01 or later
/rest/identities.xml?{query} /rest/identities.json?{query} /rest/identities.xml?{query}&attributes=<attributes_list> /rest/identities.json?{query}&attributes=<attributes_list> /rest/identity.xml?dn=<identity_dn> /rest/identity.json?dn=<identity_dn>	1.10 or later
Inputs	
/inputs.xml	1.0 or later
/rest/inputs.xml /rest/inputs.json /rest/inputs.xml?attributes=<attributes_list> /rest/inputs.json?attributes=<attributes_list>	1.10 or later
/rest/inputs.xml?installed=<true|false> /rest/inputs.json?installed=<true|false> /rest/inputs.xml?attributes=<attributes_list>&installed=<true|false> /rest/inputs.json?attributes=<attributes_list>&installed=<true|false> /rest/inputs.xml?attributes=<attributes_list>&installed=<true|false>&inuse=<true|false /rest/inputs.json?attributes=<attributes_list>&installed=<true|false>&inuse=<true|false>	1.11 or later

ACM REST API Web Endpoint	API Version
Intrusion Panels	
/bosch_intrusions.xml /bosch_intrusions.json /dmp_intrusions.xml /dmp_intrusions.json	1.12 or later
Panels	
/appliances/panels_status.xml	-
/gateways/<gateway_CN>/panels/<panel_CN>/subps/<subps_CN>/inputs.xml	1.0 or later
plasecPartitionDN parameter	1.07 or later
/rest/panels.xml /rest/panels.json /rest/panels.xml?attributes=<attributes_list> /rest/panels.json?attributes=<attributes_list>	1.10 or later
/rest/panels.xml?installed=<true|false> /rest/panels.json?installed=<true|false> /rest/panels.xml?attributes=<attributes_list>?installed=<true|false> /rest/panels.json?attributes=<attributes_list>?installed=<true|false>	1.11 or later
Roles	
/roles.xml	1.0 or later
/rest/roles.xml /rest/roles.json /rest/roles.xml?attributes=<attributes_list> /rest/roles.json?attributes=<attributes_list>	1.10 or later
/rest/roles.xml?installed=<true|false> /rest/roles.json?installed=<true|false> /rest/roles.xml?attributes=<attributes_list>?installed=<true|false> /rest/roles.json?attributes=<attributes_list>?installed=<true|false>	1.11 or later
Subpanels	
/appliances/panels_status.xml	-
/gateways/<gateway_CN>/panels/<panel_CN>/subps.xml	1.0 or later

ACM REST API Web Endpoint	API Version
`/subps.xml`	1.05 or later

`/rest/subps.xml`
`/rest/subps.json`
`/rest/subps.xml?attributes=<attributes_list>`
`/rest/subps.json?attributes=<attributes_list>`
	1.10 or later

`/rest/subps.xml?installed=<true|false>`
`/rest/subps.json?installed=<true|false>`
`/rest/subps.xml?attributes=<attributes_list>?installed=<true|false>`
`/rest/subps.json?attributes=<attributes_list>?installed=<true|false>`
	1.11 or later

Tokens	
`/tokens.xml`	1.0 or later
`plasecTokenType` parameter	1.02 or later
User Defined Lists	
`/udef_lists/userListName.xml` `/udef_lists/<user_list_name>.xml`	1.06 or later

ACC™ Software Identity Logins

For ACM and Avigilon Control Center (ACC) unification only. The ACM REST API supports transactions that retrieve the identities logged in.

> **Note:** `me.xml` is supported in ACM API version 1.07 or later.

Model

Field	Format	Definition
my_ACM_Server	String	The IP address or host name of your ACM server.

Output

Field	Format	Definition
Return code	String	The ACM system response: 200 — Success: The distinguished name (dn) identifier of the login identity 401 — Access denied

Access Groups

Access Groups are used in defining physical access to doors. They define groups of doors and that operate on a similar schedule and are used by similar people.

There are a couple caveots to adding and updating the associated Doors in an Access Group. See the examples for the coding methods required to get the Door DN's properly created.

Model: access_groups

```
cn: CN
plasecInstalled: Installed
plasecName: Name
plasecTimezoneDN: Schedule
plasecagroupDoorDN: Door DN's
```

Find All

```
# access_group_list.rb
# sample REST call to get a list of access_groups
# <twl> Avigilon, Inc.
#
# arguments: ServerAddress, ServerPort, Username, password
#
my_server = ARGV[0]
my_port = ARGV[1]
my_user = ARGV[2]
my_pwd = ARGV[3]

require 'net/https'

http = Net::HTTP.new(my_server, my_port)
http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE
```

```
http.start do |http|


# this request is for a list of all access_groups
     req = Net::HTTP::Get.new('/access_groups.xml')


     # gotta give our login and password for authorization and
     authentication
     req.basic_auth(my_user, my_pwd)


     # set the content type to xml for REST commands
     req["Content-Type"] = "application/xml"


     #send the REST request
     resp = http.request(req)


     puts "RESPONSE: #{resp.header} \n\n"
     puts "DATA: "
     puts "#{resp.body}"


end
```

Output

```
RESPONSE: #<Net::HTTPOK:0x007fd9ba36bc58>


DATA:
<?xml version="1.0" encoding="UTF-8"?>
<access-groups type="array">
     <access-group>
          <dn>cn=1,ou=agroups,cn=e3a755988b1e46bb,ou=gateways,dc=pla
          sec</dn>
          <cns type="array">
```

```
            <cn>1</cn>
      </cns>
      <objectClasses type="array">
            <objectClass>plasecAgroup</objectClass>
      </objectClasses>
      <plasecInstalled>TRUE</plasecInstalled>
      <plasecName>AccessGroup 123</plasecName>
      <plasecTimezoneDNs type="array">
            <plasecTimezoneDN>cn=2,ou=timezones,dc=plasec</plase
      cTimezoneDN>
      </plasecTimezoneDNs>
      <plasecagroupAlevelDNs type="array">
<plasecagroupAlevelDN>cn=1,ou=alevels,cn=1,ou=panels,cn=e3a755988b1e
46bb,ou=gateways,dc=plasec</plasecagroupAlevelDN>
      </plasecagroupAlevelDNs>
      <plasecagroupDNs type="array">
           <plasecagroupDN>cn=1,ou=agroups,cn=e3a755988b1e4
           6bb,ou=gateways,dc=plasec</plasecagroupDN>
           </plasecagroupDNs>
           <plasecagroupDoorDNs type="array">
<plasecagroupDoorDN>cn=1,ou=doors,cn=e3a755988b1e46bb,ou=gateways,dc
=plasec</plasecagroupDoorDN>
      </plasecagroupDoorDNs>
</access-group>
<access-group>
      <dn>cn=2,ou=agroups,cn=e3a755988b1e46bb,ou=gateways,dc=pla
      sec</dn>
      <cns type="array">
           <cn>2</cn>
      </cns>
      <objectClasses type="array">
           <objectClass>plasecAgroup</objectClass>
      </objectClasses>
```

```
        <plasecInstalled>TRUE</plasecInstalled>

        <plasecName>Employee</plasecName>

        <plasecTimezoneDNs type="array">

             <plasecTimezoneDN>cn=86502173d81648fb,ou=timezon
             es,dc=plasec</plasecTimezoneDN>

        </plasecTimezoneDNs>

        <plasecagroupAlevelDNs type="array">


    <plasecagroupAlevelDN>cn=2,ou=alevels,cn=1,ou=panels,cn=e3a755988b1e
    46bb,ou=gateways,dc=plasec</plasecagroupAlevelDN>

        </plasecagroupAlevelDNs>

        <plasecagroupDNs type="array">

             <plasecagroupDN>cn=2,ou=agroups,cn=e3a755988b1e4
             6bb,ou=gateways,dc=plasec</plasecagroupDN>

        </plasecagroupDNs>

        <plasecagroupDoorDNs type="array">


    <plasecagroupDoorDN>cn=1,ou=doors,cn=e3a755988b1e46bb,ou=gateways,dc
    =plasec</plasecagroupDoorDN>

             </plasecagroupDoorDNs>

        </access-group>

    </access-groups>
```

Create

```
# access_group_create.rb

# sample REST call to create an Access Group

# <twl> Avigilon, Inc.

#

# arguments: ServerAddress, ServerPort, Username, password, gateway_CN

#

my_server = ARGV[0]

my_port = ARGV[1]

my_user = ARGV[2]
```

```
my_pwd = ARGV[3]
my_gw = ARGV[4]

require 'net/https'

http = Net::HTTP.new(my_server, my_port)
http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE

http.start do |http|

my_url = "/access_groups.xml?gatewayid=" + my_gw
req = Net::HTTP::Post.new(my_url)
# The body of the request contains xml of the attribute(s) to update
# Note that the Door Members (plasecagroupDoorDN) can be an array of Door DN's
# and is passed in as below on an create
#
xml_string =
"<access_group>

     <plasecInstalled>TRUE</plasecInstalled>
     <plasecName>Access Group TEST4</plasecName>
     <plasecTimezoneDN>cn=2,ou=timezones,dc=plasec</plasecTimezoneDN>
     <plasecagroupDoorDN>cn=1,ou=doors,cn=#{my_
     gw},ou=gateways,dc=plasec</plasecagroupDoorDN>
     <plasecagroupDoorDN>cn=2,ou=doors,cn=#{my_
     gw},ou=gateways,dc=plasec</plasecagroupDoorDN>
     </access_group>"
req.body = xml_string

# gotta give our login and password for authorization and authentication
req.basic_auth(my_user, my_pwd)
```

```
# set the content type to xml for REST commands
req["Content-Type"] = "application/xml"
#send the REST request
resp = http.request(req)
puts "RESPONSE: #{resp.header} \n\n"
puts "DATA: "
puts "#{resp.body}"
end
```

Update

```
# access_group_update.rb
# sample REST call to update the members in a access_group
# <twl> Avigilon, Inc.
#
# arguments: ServerAddress, ServerPort, Username, password
#
my_server = ARGV[0]
my_port = ARGV[1]
my_user = ARGV[2]
my_pwd = ARGV[3]

require 'net/https'

http = Net::HTTP.new(my_server, my_port)
http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE

http.start do |http|

     # this request is to update members on a access_group
```

```
    # Since our XML document cannot have two heads we must pass the sel_
    members on the URL

    # The member DN's must be passed in the 'sel_members' attribute as
    an array

    # this must be passed on the url as show below

    #

    # Always pass a gateway_id on the URL

    #

    req =

 Net::HTTP::Put.new('/access_groups/6.xml?gateway_id=e3a755988b1e46bb&[sel_
members][]=cn=1,ou=doors,cn=e3a755988b1e46bb,ou=gateways,dc=plasec')

     # The body of the request contains xml of the attribute(s) to update

     #

    # The body must contain the <access_group> element and it cannot be
    empty.

    # So if you only want to update the members you will need to put a
    bogus

    # value in the <access_group> XML, see the example below.

    #

    xml_string = '<access_group><empty></empty></access_group>'

    req.body = xml_string

    # gotta give our login and password for authorization and
    authentication

    req.basic_auth(my_user, my_pwd)

    # set the content type to xml for REST commands

    req["Content-Type"] = "application/xml"

    #send the REST request

    resp = http.request(req)

    puts "RESPONSE: #{resp.header} \n\n"

    puts "DATA: "

    puts "#{resp.body}"

end
```

Alarms

In the ACM system certain Event Types can be defined as Alarms. Transactions for these types of events will be sent to the Alarm Monitor in the ACM system user interface and have some additional work flow. Alarms must be acknowledged and then cleared. ACM's Collaboration toolset can be used to send events in real time to remote systems. Refer to the ACM system collaboration documentation for details on setting up a collaboration. The transaction id will be part of the event sent via the collaboration.

Acknowledge Alarm

```
# sample REST call to acknowledge an alarm

# <twl> Avigilon, Inc.

#

# arguments: ServerAddress, ServerPort, Username, password

#

my_server = ARGV[0]

my_port = ARGV[1]

my_user = ARGV[2]

my_pwd = ARGV[3]


require 'net/https'


http = Net::HTTP.new(my_server, my_port)

http.use_ssl = true

http.verify_mode = OpenSSL::SSL::VERIFY_NONE


http.start do |http|


    # USEAGE: There are multiple methods in which this action may be called; for a
    single alarm or for a batch of alarms

    # for a single alarm pass the 'id' of the alarm as the id in the url

    # SPECIAL PROCESSING FOR BATCH: to acknowledge a batch of alarms pass
    the xml string
```

```
# '<batch><ack>id</ack></batch>' where <ack>id</id> is the # repeating list of id's to acknowledge.

# note that we need to get the xml passed in as a string on the URL.

# In batch mode -1 is passed as the "id" value

# NOTE: In either mode the key for alarms will be either a long integer or a GUID, likely dependent upon how you

# received the alarm records. If you are using the GUID (ie the CN) id be sure to set the param 'useguid'=TRUE

#

# to ack url = '/alarms/ack/'

# to clear url = '/alarms/clear/'

#

# Example to acknowledge a single alarm using the CN

# url = '/alarms/ack/adc907a3d7d345d2.xml?useguid=true'

#

# Example to acknowledge a batch of alarms using the CN and passing in the id's on the URL

#

url = '/alarms/ack/-1.xml?useguid=true&batch=<ack><id>9d8e2fc466004cf3</id><id>d5cda4be3d3b458d</id><id>13ec89965fc548a9</id><id>c82d10a4dc844e18</id></ack>'

#url = '/alarms/ack/-1.xml?useguid=true'


  #

req = Net::HTTP::Post.new(url)


# If just ack'ing a single alarm then do not send anything in the body

#

# The body of the request contains xml of the alarms to acknowledge

#xml_string='<batch>

# <ack>

# <id>d5cda4be3d3b458d</id>

# <id>9d8e2fc466004cf3</id>
```

```
# <id>13ec89965fc548a9</id>
# <id>c82d10a4dc844e18</id>
# </ack>
#</batch>'

#req.body = xml_string

# gotta give our login and password for authorization and authentication
req.basic_auth(my_user, my_pwd)

# set the content type to xml for REST commands
req["Content-Type"] = "application/xml"

#send the REST request
resp = http.request(req)

puts "RESPONSE: #{resp.header} \n\n"
puts "DATA: "
puts "#{resp.body}"

end
```

Clear Alarm

To clear an alarm simply change the acknowledge code URL from .../ack/... to .../clear/...

```
url = '/alarms/clear/-
1.xml?useguid=true&batch=<ack><id>9d8e2fc466004cf3</id><id>d5cda4be3d3b458d</id><id
>13ec89965fc548a9</id><id>c82d10a4dc844e18</id></ack>'
```

Appliance

Attributes of an ACM Appliance are stored in the gateway model. It is recommended that you avoid manipulation of the Appliance records via REST and instead perform any desired modifications through the ACM UI.

The gateway record defines attributes of the physical (or virtual if VMware) appliance.

Model: gateways

cn: CN

description: Description

plasecAddress: Address

plasecAltname: Alt Name

plasecId: ID

plasecInstalled: Installed

plasecLocation: Location

plasecPartitionDN: Partitions

ipNetworkNumber: IP Network Number

plasecLocalTimeStr: Time Zone

plasecName: Appliance Name

plasecgwActive: Active

plasecgwBypassmessage: Message

plasecgwDnsdomain: DNS Domain

plasecgwDnsname: Host Name

plasecgwDst: Enable Daylight Savings Time

plasecgwEdgeport: Edge Listen Port

plasecgwEmbossed2Internal: Embossed 2 Internal

plasecgwEnablebypass1: Enable Bypass 3

plasecgwEnablebypass2: Enable Bypass 2

plasecgwEnableremote: Enable Remote TCP/IP Management

plasecgwEnablesnmp: Enable SNMP

plasecgwEnableusb: Enable USB

plasecgwEventreplport: Event Replication Port

plasecgwFailoverGatewayDN: Failover Gateway DN

plasecgwKeepalive: Keep Alive

plasecgwLdapport: LDAP Connect Port

plasecgwMaxTransactions: Stored Transactions

plasecgwMercuryClientPort: Mercury Client Port

plasecgwMercuryRequireTLS: Mercury Require TLS

plasecgwNameserver: Name Server

plasecgwPartnerheartcomm: Serial

plasecgwPartnerhearttime: Heartbeat Time

plasecgwPgport: Transactions Connect Port

plasecgwRedundancyport: TCP Port

plasecgwTimeServer: Time Server

plasecgwTimezone: Schedule

plasecgwWebport: Web Server Port

plasecgwsmtpserver: SMTP Server

Find All

```
# gateway_list.rb
# sample REST call to get a list of gateways
# <twl> Avigilon, Inc.
#
# arguments: ServerAddress, ServerPort, Username, password
#
my_server = ARGV[0]
my_port = ARGV[1]
my_user = ARGV[2]
my_pwd = ARGV[3]

require 'net/https'

http = Net::HTTP.new(my_server, my_port)
http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE
```

```
http.start do |http|

     # this request is for a list of all gateways
     req = Net::HTTP::Get.new('/gateways.xml')

     # gotta give our login and password for authorization and
     authentication
     req.basic_auth(my_user, my_pwd)

     # set the content type to xml for REST commands
     req["Content-Type"] = "application/xml"

     #send the REST request
     resp = http.request(req)

     puts "RESPONSE: #{resp.header} \n\n"
     puts "DATA: "
     puts "#{resp.body}"

end
```

Create Appliance Backup

Create backup events to generate backup files of the configuration and transaction databases of the ACM system.

Model

Field	Format	Definition
gateway_id	String	The ID of the gateway.
plasecName	String	The name of the backup event.
plasecbackupType	String	The backup type.
plasecbackupSource	String	The backup source.
plasecbackupLocation	String	The name of the subdirectory where the backup files are stored.
plasecbackupEncrypt	String	TRUE or FALSE
plasecbackupHostname	String	The hostname of the backup network location.

Field	Format	Definition
plasecbackupDomainname	String	The domain name of the host.
plasecbackupPort	String	The port on the host for the backup.
plasecbackupLoginname	String	The username to log in to the backup location.
plasecbackupPassword	String	The password to log into the backup location.
day	String flag: 0 - 6	The days of the week when the backup occurs.
start_hr_min	String	The time when the backup occurs.

Output

Field	Format	Definition
Return code	String	The ACM system response: 201 — Success. 422 — Entry cannot be processed. 500 — Internal server error.

Request Example

Create appliance backup.

Note: Only the XML-based request is supported. Send an HTTP POST request.

XML

```
resp = requests.post("https://{}/gateways/{}/gw_backups.xml".format
(self.env_sconfig.get_acm(self.acm_id, "ip"), self.env_sconfig.get_
acm(self.acm_id, "gateway")),

      data=payload, headers={'Content-Type': 'application/json'},

      verify=False, auth = HTTPBasicAuth(self.env_sconfig.get_acm
(self.acm_id, "login"), self.env_sconfig.get_acm(self.acm_id,
"password")))
```

Get System Status

Note: The following requests and responses are supported in API version 1.10 or later.

Request Examples

Get the ACM system status.

XML

```
https://<my_ACM_Server>/rest/get_sys_update.xml
```

JSON

```
https://<my_ACM_Server>/rest/get_sys_update.json
```

Areas

The areas model is a reference table that defines areas known to the ACM system. Areas are primarily used for tracking the locations of people and things. For example areas can be associated with Doors to track entering and leaving an area.

Model: areas

cn: CN

plasecName: Name

plasecPartitionDN: Partitions

plasecInstalled: Enable Area

plasecareaEnforceLimit: Enforce Limit

plasecareaLogMax: Log Max

plasecareaLogMin: Log Min

plasecareaMaximum: Maximum

plasecareaMinimum: Minimum

plasecareaTwoPersons: Two Persons

Find All

GET /areas.xml

Create

```
# area_create.rb

# sample REST call to create an Area

# <twl> Avigilon, Inc.

#

# arguments: ServerAddress, ServerPort, Username, password, gateway_CN

#
```

```
my_server = ARGV[0]
my_port = ARGV[1]
my_user = ARGV[2]
my_pwd = ARGV[3]
my_gw = ARGV[4]

require 'net/https'

http = Net::HTTP.new(my_server, my_port)
http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE

http.start do |http|

my_url = "/areas.xml?gateway_id=" + my_gw
req = Net::HTTP::Post.new(my_url)

# The body of the request contains xml of the attribute(s) to update
xml_string =
"<area>
        <plasecInstalled>TRUE</plasecInstalled>
        <plasecName>Area TEST One</plasecName>
        <plasecareaEnforceLimit>TRUE</plasecareaEnforceLimit>
        <plasecareaTwoPersons>TRUE</plasecareaTwoPersons>
        <plasecareaLogMax>100</plasecareaLogMax>
        <plasecareaLogMin>2</plasecareaLogMin>
        </area>"
req.body = xml_string

# gotta give our login and password for authorization and authentication
req.basic_auth(my_user, my_pwd)
```

```
# set the content type to xml for REST commands
req["Content-Type"] = "application/xml"

#send the REST request
resp = http.request(req)

puts "RESPONSE: #{resp.header} \n\n"
puts "DATA: "
puts "#{resp.body}"

end
```

Output

```
RESPONSE: #<Net::HTTPCreated:0x007fd8928c5c40>
DATA:
```

Card Formats

The card_format reference table defines the card (badge) formats known to the system.

Model: card_formats

```
cn: CN
plasecAddress: Address
plasecInstalled: Installed
plasecName: Name
plaseccfmtCardlen: Card Number Length
plaseccfmtCardloc: Card Location
plaseccfmtEvenparitylen: Even Parity Length
plaseccfmtEvenparityloc: Even Parity Location
plaseccfmtFacilitycode: Facility Code
plaseccfmtFcodelen: Facility Code Length
plaseccfmtFcodeloc: Facility Code Location
plaseccfmtFlags: Flags
plaseccfmtIssuelen: Issue Level Length
plaseccfmtIssueloc: Issue Location
plaseccfmtManufDN: Manuf
plaseccfmtMaxbits: Max Bits
plaseccfmtMaxdigits: Max Digits
plaseccfmtMindigits: Min Digits
plaseccfmtOddparitylen: Odd Parity Length
plaseccfmtOddparityloc: Odd Parity Location
plaseccfmtOffset: Offset
plaseccfmtType: Card Format Type
```

Find All

```
GET /card_formats.xml
```

Canned Macros

Macros are commands, or sequences of commands that can control the activity of devices connected to a door, input, or output.

Model

Field	Format	Definition
cn	String	The common name.
dn	String	The distinguished name. The unique identifier of configuration data.
plasecCmacrodoorDN	String	The name of a canned macro for a door.
plasecCmacromacroDN	String	The name of a canned macro.
plasecCmacroinputDN	String	The name of a canned macro for an input.
plasecCmacrooutputDN	String	The name of a canned macro for an output.
plasecCmacrotype	String	The canned macro.
plasecTimezoneDN	String	The custom schedule.

Find All

```
# canned_macro.rb
# sample REST call to get a Canned Macro for door/input/output
# <twl> Avigilon, Inc.
#
# arguments: ServerAddress, ServerPort, Username, password
#
my_server = ARGV[0]
my_port = ARGV[1]
my_user = ARGV[2]
my_pwd = ARGV[3]

require 'net/https'

http = Net::HTTP.new(my_server, my_port)
http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE

http.start do |http|
```

```
  #
  # this request kicks off the connection
  #

  #
  # Get list of all Canned Macro that exists when no input/output/door dn is
passed
  req = Net::HTTP::Get.new("/canned_macros")

  # gotta give our login and password for authorization and authentication
  req.basic_auth(my_user, my_pwd)

  # set the content type to xml for REST commands
  req["Content-Type"] = "application/xml"

  #send the REST request
  resp = http.request(req)

  puts "HEADER: #{resp.header} \n"
  puts "CODE: #{resp.code} \n"
  puts "BODY: #{resp.body}"
  resp.each_header {|h| puts "#{h}: #{resp[h]}"}

end
```

Request Examples

Note: The following requests are supported in ACM API version 1.09 or later.

Find Canned Macro by DN

```
https://<my_ACM_Server>/canned_
macros.xml/cn=1,ou=cmacro,cn=7,ou=doors,cn=23849a3ef729444c,ou=gateways,dc=pla
sec.xml
```

Find Canned Macro by Door

```
https://<my_ACM_Server>/canned_
macros/cn=1,ou=cmacro,cn=7,ou=doors,cn=23849a3ef729444c,ou=gateways,dc=plasec.
xml
```

Find Canned Macro by Input

```
https://<my_ACM_Server>/canned_
macros/cn=1,ou=cmacro,cn=100,cn=0,cn=9,ou=panels,cn=23849a3ef729444c,ou=gatewa
ys,dc=plasec.xml
```

Response Examples

> **Note:** The following outputs are supported in ACM API version 1.09 or later.

Find All

```
<macros type="array">
<macro>

<dn>cn=1,ou=cmacro,cn=1,ou=doors,cn=421922d3d897437a,ou=gateways,dc=plasec</d
n>
<cns type="array">
<cn>1</cn>
</cns>
<entryUUID>da178b5a-2ae4-103c-92fc-75f2252cc4eb</entryUUID>
<objectClasses type="array">
<objectClass>plasecCMacro</objectClass>
</objectClasses>

<plasecCmacromacroDN>cn=513,ou=macros,cn=1,ou=panels,cn=421922d3d897437a,ou=ga
teways,dc=plasec</plasecCmacromacroDN>

<plasecCmacrooutputDN>cn=200,cn=0,cn=1,ou=panels,cn=421922d3d897437a,ou=gatewa
ys,dc=plasec</plasecCmacrooutputDN>
<plasecCmacrotype>1</plasecCmacrotype>
<plasecTimezoneDNs type="array">
<plasecTimezoneDN>cn=2,ou=timezones,dc=plasec</plasecTimezoneDN>
</plasecTimezoneDNs>
</macro>

<macro>

<dn>cn=1,ou=cmacro,cn=3,ou=doors,cn=421922d3d897437a,ou=gateways,dc=plasec</d
n>
<cns type="array">
<cn>1</cn>
</cns>
<entryUUID>9b7049818-2d15-103c-995e-71fecea7edcb</entryUUID>
<objectClasses type="array">
<objectClass>plasecCMacro</objectClass>
```

```
</objectClasses>

<plasecCmacromacroDN>cn=514,ou=macros,cn=1,ou=panels,cn=421922d3d897437a,ou=ga
teways,dc=plasec</plasecCmacromacroDN>

<plasecCmacrooutputDN>cn=200,cn=0,cn=1,ou=panels,cn=421922d3d897437a,ou=gatewa
ys,dc=plasec</plasecCmacrooutputDN>
<plasecCmacrotype>1</plasecCmacrotype>
<plasecTimezoneDNs type="array">
<plasecTimezoneDN>cn=2,ou=timezones,dc=plasec</plasecTimezoneDN>
</plasecTimezoneDNs>
</macro>
</macros>
```

Find Canned Macro by DN

```
<macros type="array">
<macro>

<dn>cn=1,ou=cmacro,cn=3,ou=doors,cn=421922d3d897437a,ou=gateways,dc=plasec</d
n>
<cns type="array">
<cn>1</cn>
</cns>
<entryUUID>da178b5a-2ae4-103c-92fc-75f2252cc4eb</entryUUID>
<objectClasses type="array">
<objectClass>plasecCMacro</objectClass>
</objectClasses>

<plasecCmacromacroDN>cn=514,ou=macros,cn=1,ou=panels,cn=421922d3d897437a,ou=ga
teways,dc=plasec</plasecCmacromacroDN>

<plasecCmacrooutputDN>cn=200,cn=0,cn=1,ou=panels,cn=421922d3d897437a,ou=gatewa
ys,dc=plasec</plasecCmacrooutputDN>
<plasecCmacrotype>1</plasecCmacrotype>
<plasecTimezoneDNs type="array">
<plasecTimezoneDN>cn=2,ou=timezones,dc=plasec</plasecTimezoneDN>
</plasecTimezoneDNs>
</macro>
</macros>
```

Collaborations

Use collaborations to pull identity information from an external database to populate identity fields in ACM, and to push identities and events from ACM to third-party applications, such as video management software.

Model

Field	Format	Definition
my_server	String	The IP address or host name of your ACM server.
my_port	String	The port number.
my_user	String	The username.
my_pwd	String	The password for the username.

Output

Field	Format	Definition
Return code	String	The ACM system response: 200 — Success 401 — Access denied

Initiate Collaboration

```
# collab_execute.rb
# sample REST call to execute a Collaboration
# <twl> Avigilon, Inc.
#
# arguments: ServerAddress, ServerPort, Username, password
#
my_server = ARGV[0]
my_port = ARGV[1]
my_user = ARGV[2]
my_pwd = ARGV[3]

require 'net/https'

http = Net::HTTP.new(my_server, my_port)
http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE
```

```
http.start do |http|

  #
  # this request kicks off a collaboration
  #

  #
  # execute an Identity one time CSV long format
  req = Net::HTTP::Post.new("/collabs/identity_collabs.xml?collabs_
dn=cn=1,ou=collaboration,dc=plasec&typ=14")

  # give our login and password for authorization and authentication
  req.basic_auth(my_user, my_pwd)

  # set the content type to xml for REST commands
  req["Content-Type"] = "application/xml"

  #send the REST request
  resp = http.request(req)

  puts "HEADER: #{resp.header} \n"
  puts "CODE: #{resp.code} \n"
  puts "BODY: #{resp.body}"
  resp.each_header {|h| puts "#{h}: #{resp[h]}"}

end
```

Doors

Doors are a primary object in the ACM system. Doors are a logical concept and typically have physical hardware associated with them. For example, a door may be an enrollment reader on a desk, a lock on a file cabinet, a turnstile or a physical door.

In addition to the create, update, read and delete (CRUD) REST commands, you can execute actions on doors and get their status.

> **Note:** Door configuration depends on the vendor. It is recommended that you add and update doors through the ACM system UI.

Model: doors

```
cn: CN
description: Description
plasecAddress: Address
plasecAltname: Alt Name
plasecId: ID
plasecInstalled: Installed
plasecLocation: Location
plasecName: Name
plasecPartitionDN: Partitions
dn: DN
ipNetworkNumber: IP address
macAddress: MAC Address
plasecAcenable: Ac Enable
plasecBatenable: Bat Enable
plasecCameraDN: Camera
plasecGateway: Appliance
plasecGrouptype: Group Type
plasecLocalTimeStr: Geographic Timezone
plasecTamperenable: Tamper Enable
plasecTimezoneDN: Custom Schedule
plasecUltamperenable: Ultamper Enable
plasecagroupRoleDN: Access Group
plasecdoorAccesscfg: Access Type
plasecdoorAccesstime: Access Time
plasecdoorAccesstimemax: Standard Access Time
plasecdoorAgroupDN: Access Groups
```

plasecdoorAltaccesstime: Extended Access
plasecdoorAlthotime: Extended Held Open Time
plasecdoorApbaction: APB Action
plasecdoorApbdelay: APB Delay
plasecdoorApbinareaDN: Into Area
plasecdoorApbmode: APB Mode
plasecdoorApboutareaDN: Out of area
plasecdoorAux1min: Aux output min
plasecdoorCustomEalevelDN: Custom Access
plasecdoorDebounce: Debounce
plasecdoorDoNotLogRextransactions: Do Not Log Rex Transactions
plasecdoorDoormode: Door mode
plasecdoorDposDN: Door Position
plasecdoorElevInputSubp1: Elevator Input Subpanel One
plasecdoorElevInputSubp2: Elevator Input Subpanel Two
plasecdoorElevInputSubp3: Elevator Input Subpanel Three
plasecdoorElevInputSubp4: Elevator Input Subpanel Four
plasecdoorElevInputSubp5: Elevator Input Subpanel Five
plasecdoorElevInputSubp6: Elevator Input Subpanel Six
plasecdoorElevInputSubp7: Elevator Input Subpanel Seven
plasecdoorElevInputSubp8: Elevator Input Subpanel Eight
plasecdoorElevOutputSubp1: Elevator Output Subpanel One
plasecdoorElevOutputSubp2: Elevator Output Subpanel Two
plasecdoorElevOutputSubp3: Elevator Output Subpanel Three
plasecdoorElevOutputSubp4: Elevator Output Subpanel Four
plasecdoorElevOutputSubp5: Elevator Output Subpanel Five
plasecdoorElevOutputSubp6: Elevator Output Subpanel Six
plasecdoorElevOutputSubp7: Elevator Output Subpanel Seven
plasecdoorElevOutputSubp8: Elevator Output Subpanel Eight
plasecdoorFacCodeEalevelDN: Facility Access
plasecdoorFdenable: Enable forced door
plasecdoorFormats: Formats
plasecdoorHdenable: Enable door held
plasecdoorHotime: Held Open Time
plasecdoorIPRdn: IPR
plasecdoorLeddrvmode: LED Mode
plasecdoorLocknum: Lock Number
plasecdoorLockfunction: Lock Function
plasecdoorMaskforced: Always Mask Forced
plasecdoorMaskforcedTimezoneDN: Masked Forced Schedule
plasecdoorMaskheld: Always Mask Held
plasecdoorMaskheldTimezoneDN: Masked Held Schedule

```
plasecdoorMercaddress: Merc Address
plasecdoorMinopen: Min Open
plasecdoorOfflineEalevelDN: Offline Access
plasecdoorOfflinedoormode: Offline Door Mode
plasecdoorOut1DN: Output
plasecdoorOut2DN: Output
plasecdoorPINSuppresstzDN: Pin Suppression
plasecdoorPINmaxdigits: PIN Max Digits
plasecdoorPINpadtype: PIN Pad Type
plasecdoorPaireddoorDN: Linked Door
plasecdoorPanel DN: Panel
plasecdoorPinattempts: Pin Attempts
plasecdoorPinlockout: Pin Lockout
plasecdoorPintimeout: Pin Timeout
plasecdoorPolicyDN: Door Policy
plasecdoorPrealarm: Held Pre-Alarm
plasecdoorRdr1DN: Reader
plasecdoorRdr2DN: Alternate Reader
plasecdoorRdrflagdecrementcnt: Decrement use counts
plasecdoorRdrflagdenyduress: Deny duress
plasecdoorRdrflagdetailedevts: Detailed events
plasecdoorRdrflagdoorforcedfilter: Door Forced Filter
plasecdoorRdrflagenableciphermode: Enable cipher mode
plasecdoorRdrflagifonlinecheckhost: If online, check host
plasecdoorRdrflaglogallaccessused: Log all access as used
plasecdoorRdrflagloggrantsrightaway: Log grants right away
plasecdoorRdrflagnohostthenprocess: No host then process
plasecdoorRdrflagnonzerocnt: Require nonzero use count
plasecdoorRdrflagnopulsedrstrikeonRex: Don't pulse door strike on REX
plasecdoorRdrflagrequirebiometricverification: Require biometric
verification
plasecdoorRdrflagrequiretwocardctrl: Require two card control
plasecdoorRdrflags: Rdr Flags
plasecdoorRdrflagsettrigflag: Set Trigger flag
plasecdoorRdrflagshuntrelay: Use Shunt Relay
plasecdoorRelayminopen: Relay Min Open
plasecdoorRex0DN: 'Rex #1'
plasecdoorRex1DN: 'Rex #2'
plasecdoorRexfunction: Don't Pulse Strike on Rex
plasecdoorRoles: Roles
plasecdoorSnowday: First card unlock
plasecdoorStrikeDN: Strike
```

plasecdoorStrikemode: Strike Mode
plasecdoorSubpDN: Subpanel
plasecdoorTimedmax: Max Timed Override
plasecdoorTyp: Door Type
plasecdoorTzdoormode: Custom Mode
plasecdoorUseTracking: Door use Tracking
plasecgroupPolicyDN: Group Policy
plasecidentityCameraType: Camera Type
plasecinputMask: Masked
plasecpanelAcceptport: Accept Port
plasecpanelAlthostip: Althost IP
plasecpanelAltportcomms: Altport Comms
plasecpanelAltportenable: Alternate comms
plasecpanelAltportipaddr: Alt IP address
plasecpanelAltportnetretry: Alt reply timeout
plasecpanelAltportreplyto: Alt poll delay
plasecpanelBaud: Baud
plasecpanelCTrx: C-Trx
plasecpanelCaccesslevels: C-Access Levels
plasecpanelCcards: C-Cards
plasecpanelCholidays: C-Holidays
plasecpanelCommtyp: Comm Type
plasecpanelCports: C-Ports
plasecpanelCtimezones: C-Schedules
plasecpanelEncrypt: Encrypt
plasecpanelGmzoffset: GMZ Offset
plasecpanelHostip: Host IP
plasecpanelInput1enable: Input1 Enable
plasecpanelInput1masked: Input1 Masked
plasecpanelInput2enable: Input2 Enable
plasecpanelInput2masked: Input2 Masked
plasecpanelListenport: TCP port
plasecpanelManuf: Vendor
plasecpanelMkey1: M-key1
plasecpanelMkey2: Mkey 2
plasecpanelModel: Model
plasecpanelNetretry: Reconnect
plasecpanelOfflineto: Offline timeout
plasecpanelRelay1duration: Relay1 Duration
plasecpanelRelay1mode: Relay 1 Mode
plasecpanelRelay2duration: Relay2 Duration
plasecpanelRelay2mode: Relay 2 Mode

```
plasecpanelReplyto: Reply timeout
plasecpanelRtscontrol: Rts Control
plasecpanelTimeoutdiscon: Timeout Discon
```

Door Commands

```
# door_commands.rb
# sample REST call to perform actions on a door
# <twl> Avigilon, Inc.
#
# arguments: ServerAddress, ServerPort, Username, password
#
my_server = ARGV[0]
my_port = ARGV[1]
my_user = ARGV[2]
my_pwd = ARGV[3]

require 'net/https'

http = Net::HTTP.new(my_server, my_port)
http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE

http.start do |http|


# this request sends a command to a door
# possible door commands include: grant, lock, unlock, restore,
policy, disable, unmask_forced, mask_forced, unmask_held, mask_held
# the full door DN must be passed as part of the URL
req = Net::HTTP::Post.new
('/doors/cn=1,ou=doors,cn=e3a755988b1e46bb,ou=gateways,dc=plasec/res
tore.xml')

# gotta give our login and password for authorization and
authentication
req.basic_auth(my_user, my_pwd)
# set the content type to xml for REST commands
req["Content-Type"] = "application/xml"
#send the REST request
resp = http.request(req)
puts "RESPONSE: #{resp.header} \n\n"
```

```
    puts "DATA: "
    puts "#{resp.body}"
end
```

Panel Change

For Schlage IP wireless devices.

An ENGAGE gateway is a panel that manages linked devices, which are externally commissioned IP wireless locks or remote undogging (RU) and remote monitoring (RM) devices.

panel

```
cn: CN
dn: Distinguished Name
entryUUID: Universally Unique Identifier of the entry
ipNetworkNumber: IP Network Number
plasecAddress: Address
plasecInstalled: Installed
plasecLocalTimeStr: Timezone
plasecName: Name
plasecpanelCommtyp: Comm Type
plasecpanelCredential: Gateway Site Credential
plasecpanelGmzoffset: GMZ Offset
plasecpanelManuf: Vendor
plasecpanelModel: Model
plasecpanelOfflineto: Offline timeout
plasecpanelRequireTLS: TLS Required
plasecpanelSchlageSite: Gateway Site
plasecPartitionDN: Partitions
plasecVersion: Version
```

linked-device

```
cn: CN
dn: Distinguished Name
entryUUID: Universally Unique Identifier of the entry
macAddress: MAC Address
plasecLinkedDeviceLinkId: Link ID of Linked Device Link
plasecLinkedDeviceModelType: Model Type of Linked Device
plasecLinkedDeviceTokenCount: Token Count of Linked Device
plasecName: Name
```

Output Examples

Note: The following output is supported in ACM API version 1.03 or later.

```
<?xml version="1.0" encoding="UTF-8"?>
<panel-info>
<panel>
    <dn>cn=4,ou=panels,cn=ada363e86d3e42f1,ou=gateways,dc=plasec</d
n>
    <cns type="array">
      <cn>4</cn>
    </cns>
<entryUUID>595521b0-1f8a-103a-8b2d-9f72a4245069</entryUUID>
<ipNetworkNumber>nn.nn.nnn.nnn</ipNetworkNumber>
<objectClasses type="array">
<objectClass>plasecPanel</objectClass>
</objectClasses>
<plasecAddress>4</plasecAddress>
<plasecInstalled>FALSE</plasecInstalled>
<plasecLocalTimeStr>Canada/Pacific</plasecLocalTimeStr>
<plasecName>gwe-36DF</plasecName>
<plasecPartitionDNs type="array">
<plasecPartitionDN>cn=6cfdbc67bf6e4018,ou=partitions,dc=plasec</plas
ecPartitionDN>
</plasecPartitionDNs>
<plasecVersion>1</plasecVersion>
<plasecpanelCommtyp>4</plasecpanelCommtyp>
<plasecpanelCredential>Basic abcdef123456</plasecpanelCredential>
<plasecpanelGmzoffset>-28800</plasecpanelGmzoffset>
<plasecpanelManufs type="array">
<plasecpanelManuf>cn=10,ou=manufs,dc=plasec</plasecpanelManuf>
</plasecpanelManufs>
<plasecpanelModel>1</plasecpanelModel>
<plasecpanelOfflineto>15000</plasecpanelOfflineto>
<plasecpanelRequireTLS>TRUE</plasecpanelRequireTLS>
<plasecpanelSchlageSite>cn=ada2afc0ff7b4ae2,ou=sites,cn=10,ou=manufs
,dc=plasec</plasecpanelSchlageSite>
</panel>
<door-subps nil="true"/>
<linked-devices type="array">
<linked-device>
```

```
<dn>cn=a100000000002327,ou=linkedDevices,cn=4,ou=panels,cn=ada363e86
d3e42f1,ou=gateways,dc=plasec</dn>
<cns type="array">
<cn>a100000000002327</cn>
</cns>
<entryUUID>32a5d9f4-1f96-103a-8b39-9f72a4245069</entryUUID>
<macAddresses type="array">
<macAddress>nn:nn:nn:nn:nn:nn</macAddress>
</macAddresses>
<objectClasses type="array">
<objectClass>plasecLinkedDevice</objectClass>
</objectClasses>
<plasecLinkedDeviceLinkId>dev00000</plasecLinkedDeviceLinkId>
<plasecLinkedDeviceModelType>nde</plasecLinkedDeviceModelType>
<plasecLinkedDeviceTokenCount>0</plasecLinkedDeviceTokenCount>
<plasecName>NDE-1234</plasecName>
</linked-device>
</linked-devices>
</panel-info>
```

Door Elevator Grant

If the door is an elevator, then we are required to send the floor number that the Grant should apply to.

```
# door_grant_elev.rb
# sample REST call to send a grant command to an Elevator Door
# <twl> Avigilon, Inc.
#
# arguments: ServerAddress, ServerPort, Username, password
#
my_server = ARGV[0]
my_port = ARGV[1]
my_user = ARGV[2]
my_pwd = ARGV[3]

require 'net/https'

http = Net::HTTP.new(my_server, my_port)
http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE

http.start do |http|
```

```
# this request is to send a grant to an Elevator Door
# the Elevator Floor must be passed on the URL
#
req = Net::HTTP::Post.new
('/doors/cn=4,ou=doors,cn=263379c434ea4ff8,ou=gateways,dc=plasec/gra
nt.xml?elev_floor=3')

# gotta give our login and password for authorization and
authentication
req.basic_auth(my_user, my_pwd)
# set the content type to xml for REST commands
req["Content-Type"] = "application/xml"


#send the REST request
resp = http.request(req)


puts "RESPONSE: #{resp.header} \n\n"
puts "DATA: "
puts "#{resp.body}"


end
```

Find All

```
# door_list.rb
# sample REST call to get a list of doors
# <twl> Avigilon, Inc.
#
# arguments: ServerAddress, ServerPort, Username, password
#
my_server = ARGV[0]
my_port = ARGV[1]
my_user = ARGV[2]
my_pwd = ARGV[3]


require 'net/https'


http = Net::HTTP.new(my_server, my_port)
http.use_ssl = true
```

```
    http.verify_mode = OpenSSL::SSL::VERIFY_NONE


    http.start do |http|


    # this request is for a list of all doors
    req = Net::HTTP::Get.new('/doors.xml')


    # gotta give our login and password for authorization and
    authentication
    req.basic_auth(my_user, my_pwd)


    # set the content type to xml for REST commands
    req["Content-Type"] = "application/xml"


    #send the REST request
    resp = http.request(req)


    puts "RESPONSE: #{resp.header} \n\n"
    puts "DATA: "
    puts "#{resp.body}"


end
```

Door Status

```
    # door_status_xml.rb
    # sample REST call to get the status for a list of doors
    #
    # This program will query the ACM system and return the door status
    information for the
    # doors passed in via the 'door_dns' parameter.
    # Note that this can be called to return data in either JSON format
    or XML
    #
    # <twl> Avigilon, Inc.
    #
```

```
    # arguments: ServerAddress, ServerPort, Username, password
    #
    my_server = ARGV[0]
    my_port = ARGV[1]
    my_user = ARGV[2]
    my_pwd = ARGV[3]


    require 'net/https'


    http = Net::HTTP.new(my_server, my_port)
    http.use_ssl = true
    http.verify_mode = OpenSSL::SSL::VERIFY_NONE


    http.start do |http|


    # this request is for the door status for the listed doors
    req = Net::HTTP::Get.new("/doors/show_status.xml?")
    #
    # pass in the DNs of the doors that we want to grab the status for
    # this should be in the format as shown below, full DNs separated by
    slashes "/" and encased with the tag
    # <door_dns> </door_dns>
    #
    xml_string = "<door_
    dns>cn=1,ou=doors,cn=e3a755988b1e46bb,ou=gateways,dc=plasec/cn=2,ou=
    doors,cn=e3a755988b1e46bb,ou=gateways,dc=plasec/cn=3,ou=doors,cn=e3a
    755988b1e46bb,ou=gateways,dc=plasec</door_dns>"
    req.body = xml_string
    # gotta give our login and password for authorization and
    authentication
    req.basic_auth(my_user, my_pwd)
    # set the content type to xml for REST commands
    req["Content-Type"] = "application/xml"
    #send the REST request
    resp= http.request(req)
    puts "HEADER: #{resp.header} \n\n"
    puts "DATA: "
    puts "#{resp.body}"
end
```

Response Examples

In the output you can see that there are two arrays, mode and stats. Mode is an array showing the current mode for each door. Stats is an array of doors, for each door you will get a status that matches the doors status page in the UI. The seven statuses will be one of ‘normal’, ‘trouble’ or ‘alarm’. The values are for; Communications, Lock, Power, Tamper, Battery, Forced and Held. The last value will indicate the current door position if available, typically this is ‘Open’ or ‘Closed’.

> **Note:** The following output is supported in ACM API version 1.02 or earlier.

```
HEADER: #<Net::HTTPOK:0x007f9b390b6068>
DATA:
<?xml version="1.0" encoding="UTF-8"?>
<hash>
<doors>
     <type>door</type>
     <modes>
     <1-e3a755988b1e46bb>Card or Pin</1-e3a755988b1e46bb>
     <2-e3a755988b1e46bb></2-e3a755988b1e46bb>
     <3-e3a755988b1e46bb></3-e3a755988b1e46bb>
</modes>
<stats>
     <1-e3a755988b1e46bb type="array">
     <1-e3a755988b1e46bb>normal</1-e3a755988b1e46bb>
     <1-e3a755988b1e46bb>normal</1-e3a755988b1e46bb>
     <1-e3a755988b1e46bb>normal</1-e3a755988b1e46bb>
     <1-e3a755988b1e46bb>normal</1-e3a755988b1e46bb>
     <1-e3a755988b1e46bb>normal</1-e3a755988b1e46bb>
     <1-e3a755988b1e46bb>normal</1-e3a755988b1e46bb>
     <1-e3a755988b1e46bb>normal</1-e3a755988b1e46bb>
     <1-e3a755988b1e46bb>Closed</1-e3a755988b1e46bb>
</1-e3a755988b1e46bb>
<2-e3a755988b1e46bb type="array">
     <2-e3a755988b1e46bb>alarm</2-e3a755988b1e46bb>
     <2-e3a755988b1e46bb>normal</2-e3a755988b1e46bb>
     <2-e3a755988b1e46bb>normal</2-e3a755988b1e46bb>
     <2-e3a755988b1e46bb>normal</2-e3a755988b1e46bb>
     <2-e3a755988b1e46bb>normal</2-e3a755988b1e46bb>
     <2-e3a755988b1e46bb>normal</2-e3a755988b1e46bb>
```

```
     <2-e3a755988b1e46bb>normal</2-e3a755988b1e46bb>
     <2-e3a755988b1e46bb>Closed</2-e3a755988b1e46bb>
</2-e3a755988b1e46bb>
<3-e3a755988b1e46bb type="array">
     <3-e3a755988b1e46bb>alarm</3-e3a755988b1e46bb>
     <3-e3a755988b1e46bb>normal</3-e3a755988b1e46bb>
     <3-e3a755988b1e46bb>normal</3-e3a755988b1e46bb>
     <3-e3a755988b1e46bb>normal</3-e3a755988b1e46bb>
     <3-e3a755988b1e46bb>normal</3-e3a755988b1e46bb>
     <3-e3a755988b1e46bb>normal</3-e3a755988b1e46bb>
     <3-e3a755988b1e46bb>normal</3-e3a755988b1e46bb>
     <3-e3a755988b1e46bb>Closed</3-e3a755988b1e46bb>
</3-e3a755988b1e46bb>
</stats>
</doors>
</hash>
```

Note: The following output is supported in API version 1.04 or later.

```
<doors>
     <type>door</type>
     <modes type="array">
          <mode type="array">
          <mode>cn=1,ou=doors,cn=73fd33c21b77499a,ou=gatew
          ays,dc=plassec</mode>
          <mode>Card Only</mode>
          </mode>
     </modes>
     <stats type="array">
          <stat>
               <dn>cn=1,ou=doors,cn=73fd33c21b77499a,
               ou=gateways,dc=plassec</dn>
               <status type="array">
                    <status>normal</status>
                    <status>normal</status>
```

```
                        <status>normal</status>
                        <status>normal</status>
                        <status>normal</status>
                        <status>normal</status>
                        <status>normal</status>
                        <status/>
                    </status>
                </stat>
            </stats>
        </doors>
```

Get Installed or Uninstalled Doors

Note: The following requests and responses are supported in API version 1.11 or later.

Request and Response Examples

Get all the installed doors by specifying door attributes.

XML

Request:

```
https://<my_ACM_
Server>/rest/doors.xml?attributes=dn,plasecName,plasecInstalled,plasecPanelMan
uf,entryUUID,plasecdoorPanelDN,plasecdoorAccesscfg,plasecPartitionDN&installed
=true
```

Response:

```
<doors>
  <door>
    <dn>cn=704,ou=doors,cn=4aebfacfa2624c01,ou=gateways,dc=plasec</dn>
    <plasecInstalled>TRUE</plasecInstalled>
    <plasecName>Door-613-1</plasecName>
    <plasecpanelManuf>cn=1,ou=manufs,dc=plasec</plasecpanelManuf>

<plasecdoorPanelDN>cn=613,ou=panels,cn=4aebfacfa2624c01,ou=gateways,dc=plasec<
/plasecdoorPanelDN>
    <plasecdoorAccesscfg>0</plasecdoorAccesscfg>
    <entryUUID>66f3f7fe-253e-103c-9594-ff0eded78b54</entryUUID>
  </door>
```

```
  <door>
    <dn>cn=705,ou=doors,cn=4aebfacfa2624c01,ou=gateways,dc=plasec</dn>
    <plasecInstalled>TRUE</plasecInstalled>
    <plasecName>Door-614-0</plasecName>
    <plasecpanelManuf>cn=1,ou=manufs,dc=plasec</plasecpanelManuf>

<plasecdoorPanelDN>cn=614,ou=panels,cn=4aebfacfa2624c01,ou=gateways,dc=plasec<
/plasecdoorPanelDN>
    <plasecdoorAccesscfg>0</plasecdoorAccesscfg>
    <entryUUID>696ab054-253e-103c-959f-ff0eded78b54</entryUUID>
    <plasecPartitionDNs type="array">

<plasecPartitionDN>cn=7781a327f9e6413d,ou=partitions,dc=plasec</plasecPartitio
nDN>
    </plasecPartitionDNs>
  </door>
  …
</doors>
```

JSON

Request:

```
https://<my_ACM_
Server>/rest/doors.json?attributes=dn,plasecName,plasecInstalled,plasecPanelMa
nuf,entryUUID,plasecdoorPanelDN,plasecdoorAccesscfg,plasecPartitionDN&installe
d=true
```

Response:

```
[{\"dn\":\"cn=12,ou=doors,cn=4aebfacfa2624c01,ou=gateways,dc=plasec\",\"plasec
Name\":\"Mr62e\",\"plasecpanelManuf\":\"cn=1,ou=manufs,dc=plasec\",\"plasecdoo
rPanelDN\":\"cn=7,ou=panels,cn=4aebfacfa2624c01,ou=gateways,dc=plasec\",\"plas
ecdoorAccesscfg\":\"0\",\"entryUUID\":\"d536205c-e403-103a-81f8-
73a819c97040\",\"plasecInstalled\":\"TRUE\",\"plasecPartitionDN\":\"cn=7781a32
7f9e6413d,ou=partitions,dc=plasec\"},
{\"dn\":\"cn=6,ou=doors,cn=4aebfacfa2624c01,ou=gateways,dc=plasec\",\"plasecNa
me\":\"HID\",\"plasecpanelManuf\":\"cn=2,ou=manufs,dc=plasec\",\"plasecInstall
ed\":\"TRUE\",\"plasecdoorPanelDN\":\"cn=2,ou=panels,cn=4aebfacfa2624c01,ou=ga
teways,dc=plasec\",\"plasecdoorAccesscfg\":\"0\",\"entryUUID\":\"847bdf42-
2073-103b-9a84-e3e1e4dc2a75\"},
{\"dn\":\"cn=7,ou=doors,cn=4aebfacfa2624c01,ou=gateways,dc=plasec\",\"plasecNa
me\":\"MERCURY_
2\",\"plasecpanelManuf\":\"cn=1,ou=manufs,dc=plasec\",\"plasecInstalled\":\"TR
UE\",\"plasecdoorPanelDN\":\"cn=7,ou=panels,cn=4aebfacfa2624c01,ou=gateways,dc
=plasec\",\"plasecdoorAccesscfg\":\"0\",\"entryUUID\":\"9a2a0d64-2073-103b-
9a98-e3e1e4dc2a75\"}]
```

Delegations

A delegation is a list of permissions to access specific functionality within the ACM system that can be assigned to an ACM operator (or a group of operators) using roles.

Model

Field	Format	Definition
cn	String	The common name.
dn	String	The distinguished name. The unique identifier of configuration data.
plasecCmacrodoorDN	String	The name of a canned macro for a door.
plasecCmacromacroDN	String	The name of a canned macro.
plasecCmacroinputDN	String	The name of a canned macro for an input.
plasecCmacrooutputDN	String	The name of a canned macro for an output.
plasecCmacrotype	String	The canned macro.
plasecTimezoneDN	String	The custom schedule.

Get Delegations

Note: The following requests and responses are supported in API version 1.10 or later.

Request Examples

Get all the delegations with the specified attributes.

XML

```
https://<my_ACM_Server>/rest/right_
groups.xml?attributes=dn,plasecName,plasecrightgroupRightDN,plasecPartitionDN
```

JSON

```
https://<my_ACM_Server>/rest/right_
groups.json?attributes=dn,plasecName,plasecrightgroupRightDN,plasecPartitionDN
```

Event Types

Events in the ACM system are described by two models Event Types and Points known as Events in the UI. There is a one to many relationship between these two models with Event Types having many Points. You would not typically add a new Event Type but you may want to update attributes such as the instructions or email address's that get notification when the Event Type occurs.

Model: evtypes

```
cn: CN

plasecName: Name

plasecetypAlarm: Alarm

plasecetypEmail: Email

plasecetypInstructions: Instructions

plasecetypLogged: Logged

plasecetypMasked: Masked

plasecetypPriority: Priority

plasecetypSuppressTz: Suppress Schedule
```

Find All

```
# evtype_list.rb

# sample REST call to get a list of Event Types

# <twl> Avigilon, Inc.

#

# arguments: ServerAddress, ServerPort, Username, password

#

my_server = ARGV[0]

my_port = ARGV[1]

my_user = ARGV[2]

my_pwd = ARGV[3]


require 'net/https'
```

```
http = Net::HTTP.new(my_server, my_port)
http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE

http.start do |http|

     # this request is for a list of all event types
     req = Net::HTTP::Get.new('/evtypes.xml')
     # gotta give our login and password for authorization and
     authentication
     req.basic_auth(my_user, my_pwd)
     # set the content type to xml for REST commands
     req["Content-Type"] = "application/xml"
     #send the REST request
     resp = http.request(req)
     puts "RESPONSE: #{resp.header} \n\n"
     puts "DATA: "
     puts "#{resp.body}"
end
```

Update

```
# evtype_update.rb
# sample REST call to update an Event Type record
# <twl> Avigilon, Inc.
#
# arguments: ServerAddress, ServerPort, Username, password, evtype_cn
#
my_server = ARGV[0]
my_port = ARGV[1]
my_user = ARGV[2]
```

```
my_pwd = ARGV[3]
my_evtype_cn = ARGV[4]

require 'net/https'

http = Net::HTTP.new(my_server, my_port)
http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE

http.start do |http|

     # this request is to update an Event Type
      my_url = '/evtypes/' + my_evtype_cn + '.xml'
      req = Net::HTTP::Put.new(my_url)

     # The body of the request contains xml of the attribute(s)
     to update
     # in this case we're adding instructions to the Event Type
     xml_string = '<evtype>
          <plasecetypInstructions>Guard: Physically go and
          check to see why this door is being held open.
          Enter notes to indicate the time of the check
          and what was found.</plasecetypInstructions>
     </evtype>'

     req.body = xml_string

     # gotta give our login and password for authorization and
     authentication
     req.basic_auth(my_user, my_pwd)

     # set the content type to xml for REST commands
     req["Content-Type"] = "application/xml"
```

```
    #send the REST request
    resp = http.request(req)

    puts "RESPONSE: #{resp.header} \n\n"
    puts "DATA: "
    puts "#{resp.body}"

end
```

Events

Model: points

cn: CN

plasecAddress: Address

plasecInstalled: Installed

plasecName: Name

plasecResourceType: Resource Type

plasecTimezoneDN: Suppress Time

plasecpointAlarmSound: Alarm Sound

plasecpointEmail: Email

plasecpointEvtype: Event Type

plasecpointHasreturn: Has on/off

plasecpointInstruction: Instructions

plasecpointLogged: Logged

plasecpointMasked: Masked

plasecpointPriority: Priority

plasecpointProc: Procedure Id

plasecpointRetransmit: Re-transmit

plasecpointRoleDN: Role

plasecpointRtnevtype: Return Event

plasecpointRtnname: Return Name

plasecpointRtnpriority: Return Priority

plasecpointRtnretransmit: Return Re-transmit

plasecpointShowVideo: Show Video

plasecpointSourcetype: Source Type

plasecpointSourcetypePoint: Source Type

plasecpointTwoPerson: Two Person Required to Clear

Find All

```
/points.xml
```

Global Actions

Global Actions are often used and powerful feature of the ACM system. By defining Global Actions users are able to create user defined actions that can perform a wide variety of tasks within ACM such as locking down the entire organization, firing outputs, sending email or unlocking doors. Global Actions can be complicated to setup so it is recommended that you create and update Global Actions through the ACM system UI. If you must create and update global actions via REST, refer to the ACM system UI for the values of the Type and Sub-Type as new features are regularly added.

Model: ioactions

```
cn: cn
plasecAddress: Address
plasecId: ID
plasecInstalled: Installed
plasecLocation: Location
plasecName: Name
plasecActionEmailAddresses: Email Address
plasecActionMacroDN: Macro
plasecActionQualifier: Sub-Type
plasecActionSoftTriggerDN: Trigger
plasecActionType: Type
```

Find All

```
#global_action_list.rb
# sample REST call to get a list of global_actions
# <twl> Avigilon, Inc.
#
# arguments: ServerAddress, ServerPort, Username, password
#
my_server = ARGV[0]
my_port = ARGV[1]
```

```
my_user = ARGV[2]
my_pwd = ARGV[3]

require 'net/https'

http = Net::HTTP.new(my_server, my_port)
http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE

http.start do |http|

     # this request is for a list of all global_actions
     req = Net::HTTP::Get.new('/ioactions.xml')

     # gotta give our login and password for authorization and
     authentication
     req.basic_auth(my_user, my_pwd)

     # set the content type to xml for REST commands
     req["Content-Type"] = "application/xml"

     #send the REST request
     resp = http.request(req)

     puts "RESPONSE: #{resp.header} \n\n"
     puts "DATA: "
     puts "#{resp.body}"

end
```

Create

```
# global_action_create.rb
# sample REST call to create a global action
# <twl> Avigilon, Inc.
#
# arguments: ServerAddress, ServerPort, Username, password
#
my_server = ARGV[0]
my_port = ARGV[1]
my_user = ARGV[2]
my_pwd = ARGV[3]
my_gw_cn = 'e3a755988b1e46bb'

require 'net/https'
http = Net::HTTP.new(my_server, my_port)
http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE

http.start do |http|

    #
    # The member DN's must be passed in the 'sel_members'
    attribute as an array as parameters on the URL
    # The gateway(appliance) for this global action must be
    passed on the url
    #
    url = '/ioactions.xml' + '?[sel_members]
    []=cn=4,ou=doors,cn=' + my_gw_cn +',ou=gateways,dc=plasec&
    [sel_members[]=cn=3,ou=doors,cn=' + my_gw_cn +
    ',ou=gateways,dc=plasec' + '&gatewayid=' + my_gw_cn

    req = Net::HTTP::Post.new(url)
```

```
# The body of the request contains xml of the attribute(s)
to update

#

xml_string =

"<ioaction>

     <plasecName>TEST GLOBAL ACTION</plasecName>

     <plasecActionType>4</plasecActionType>

     <plasecActionQualifier>2</plasecActionQualifier>

     <plasecActionSoftTriggerDN></plasecActionSoftTri
     ggerDN>

     <plasecActionMacroDN></plasecActionMacroDN>

     <plasecActionEmailAddresses></plasecActionEmailA
     ddresses>

     <plasecActionLibraryType>1</plasecActionLibraryT
     ype>

</ioaction>"


req.body = xml_string


# gotta give our login and password for authorization and
authentication

req.basic_auth(my_user, my_pwd)


# set the content type to xml for REST commands

req["Content-Type"] = "application/xml"


#send the REST request

resp = http.request(req)


puts "RESPONSE: #{resp.header} \n\n"

puts "DATA: "

puts "#{resp.body}"

end
```

Update

```
# global_action_update.rb
# sample REST call to update a global action
# <twl> Avigilon, Inc.
#
# arguments: ServerAddress, ServerPort, Username, password
#
my_server = ARGV[0]
my_port = ARGV[1]
my_user = ARGV[2]
my_pwd = ARGV[3]
my_gw_cn = 'e3a755988b1e46bb'

require 'net/https'

http = Net::HTTP.new(my_server, my_port)
http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE
http.start do |http|

    #
    # The member DN's must be passed in the 'sel_members'
    attribute as an array as parameters on the URL
    # The gateway(appliance) for this global action must be
    passed on the url
    #
    reqstring = '/ioactions/cn=4,ou=actions,cn=' + my_gw_cn +
    ',ou=gateways,dc=plasec.xml' + '?[sel_members]
    []=cn=16,ou=doors,cn=' + my_gw_cn +
    ',ou=gateways,dc=plasec&[sel_members[]=cn=17,ou=doors,cn='
    + my_gw_cn + ',ou=gateways,dc=plasec' + '&gatewayid=' +
```

```
my_gw_cn


req = Net::HTTP::Put.new(reqstring)

# The body of the request contains xml of the attribute(s)
to update

#

xml_string =

"<ioaction>

     <plasecName>TEST GLOBAL ACTION-4</plasecName>

     <plasecActionType>4</plasecActionType>

     <plasecActionQualifier>2</plasecActionQualifier>

     <plasecActionSoftTriggerDN></plasecActionSoftTri
     ggerDN>

     <plasecActionMacroDN></plasecActionMacroDN>

     <plasecActionEmailAddresses></plasecActionEmailA
     ddresses>

     <plasecActionLibraryType>1</plasecActionLibraryT
     ype>

</ioaction>"


req.body = xml_string


# gotta give our login and password for authorization and
authentication

req.basic_auth(my_user, my_pwd)


# set the content type to xml for REST commands

req["Content-Type"] = "application/xml"


#send the REST request

resp = http.request(req)
```

```
    puts "RESPONSE: #{resp.header} \n\n"
    puts "DATA: "
    puts "#{resp.body}"

end
```

Execute

```
# global_action_execute.rb
# sample REST call to execute a global action
# <twl> Avigilon, Inc.
#
# arguments: ServerAddress, ServerPort, Username, password
#
my_server = ARGV[0]
my_port = ARGV[1]
my_user = ARGV[2]
my_pwd = ARGV[3]

require 'net/https'

http = Net::HTTP.new(my_server, my_port)
http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE

http.start do |http|

    req = Net::HTTP::Post.new
    ('/ioactions/cn=2,ou=actions,cn=e3a755988b1e46bb,ou=gatewa
    ys,dc=plasec/execute.xml')

    # gotta give our login and password for authorization and
```

```
    authentication
    req.basic_auth(my_user, my_pwd)

    # set the content type to xml for REST commands
    req["Content-Type"] = "application/xml"

    #send the REST request
    resp = http.request(req)

    puts "RESPONSE: #{resp.header} \n\n"
    puts "DATA: "
    puts "#{resp.body}"

end
```

Groups

Groups in the ACM system are a generic way of associating similar items together for processing in various locations in the system. The ‘member’ attribute contains the DN’s of the members of the Group. In general you should ensure that you group like objects together. For example do not create a group that has DN’s for both Identities and Doors. Groups require a bit of special coding during update to enable correct processing of the member attribute. Note how we can send multiple member attributes during create and we must send the member values on the URL during updates.

Model: groups

```
cn: CN

plasecName: Name

plasecGrouptype: Group Type

plasecgroupPolicyDN: Policy

member: Member DN’s
```

Find All

```
GET /groups.xml
```

Create

```
# group_add.rb

# sample REST call to add a new group

# <twl> Avigilon, Inc.

#

# arguments: ServerAddress, ServerPort, Username, password

#

my_server = ARGV[0]

my_port = ARGV[1]

my_user = ARGV[2]

my_pwd = ARGV[3]
```

```
require 'net/https'


http = Net::HTTP.new(my_server, my_port)

http.use_ssl = true

http.verify_mode = OpenSSL::SSL::VERIFY_NONE


http.start do |http|


     req = Net::HTTP::Post.new('/groups.xml')


     # The body of the request contains xml of the attribute(s)
     to update

     xml_string =

     "<group>

          <plasecName>TEST GROUP FIVE</plasecName>

          <member>cn=1,ou=doors,cn=e3a755988b1e46bb,ou=gat
          eways,dc=plasec</member>

          <member>cn=3,ou=doors,cn=e3a755988b1e46bb,ou=gat
          eways,dc=plasec</member>

          </group>"

     req.body = xml_string


     # gotta give our login and password for authorization and
     authentication

     req.basic_auth(my_user, my_pwd)


     # set the content type to xml for REST commands

     req["Content-Type"] = "application/xml"


     #send the REST request

     resp = http.request(req)
```

```
        puts "RESPONSE: #{resp.header} \n\n"
        puts "DATA: "
        puts "#{resp.body}"
end
```

Update

Some special coding is required to update the members of the Group. Because our XML document cannot have two heads we need to pass the members as an array of DN's on the URL.

```
# group_update.rb
# sample REST call to update the members in a group
# <twl> Avigilon, Inc.
#
# arguments: ServerAddress, ServerPort, Username, password
#
my_server = ARGV[0]
my_port = ARGV[1]
my_user = ARGV[2]
my_pwd = ARGV[3]

require 'net/https'

http = Net::HTTP.new(my_server, my_port)
http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE

http.start do |http|

    # this request is to update members on a group
    # Since our XML document cannot have two heads we must
```

```
pass the sel_members on the URL

# The member DN's must be passed in the 'sel_members'
attribute as an array

# this must be passed on the url as show below

# A parameter 'mode=members' must be passed also to tell
the system to update the members attribtue

# req = Net::HTTP::Put.new
('/groups/4f6d4c48e6d54b81.xml?mode=members&[sel_members]
[]=cn=1,ou=doors,cn=e3a755988b1e46bb,ou=gateways,dc=plase
c&[sel_members]
[]=cn=2,ou=doors,cn=e3a755988b1e46bb,ou=gateways,dc=plasec')

# The body of the request contains xml of the attribute(s)
to update

# if the sel_members array is passed on the url then we
can send updates to other attributes in the body

xml_string = '<group>

     <plasecName>Test Door2</plasecName>

</group>'


req.body = xml_string


# gotta give our login and password for authorization and
authentication

req.basic_auth(my_user, my_pwd)


# set the content type to xml for REST commands

req["Content-Type"] = "application/xml"


#send the REST request

resp = http.request(req)


puts "RESPONSE: #{resp.header} \n\n"

puts "DATA: "

puts "#{resp.body}"
```

```
end
```

Holidays

The holiday model stores reference data for holidays which can have special access control processing requirements.

Model: holidays

```
cn: CN

plasecName: Name

plasecholDate: Date

plasecholLength: Additional Days

plasecholPreserveSchedDay: Preserve schedule days

plasecholType: Type
```

Find All

```
GET /holidays.xml
```

Create

```
# holiday_create.rb

# sample REST call to create a new holidays

# <twl> Avigilon, Inc.

#

# arguments: ServerAddress, ServerPort, Username, password

#

my_server = ARGV[0]

my_port = ARGV[1]

my_user = ARGV[2]

my_pwd = ARGV[3]


require 'net/https'
```

```
http = Net::HTTP.new(my_server, my_port)

http.use_ssl = true

http.verify_mode = OpenSSL::SSL::VERIFY_NONE

http.start do |http|


     #

     # Special processing on Holidays, we must pass the
     plasecholDate value on the url rather than in the xml
     stream

     #

     req = Net::HTTP::Post.new
     ('/holidays.xml?plasecholDate=1/1/2021')


     # The body of the request contains xml of the attribute(s)
     to update

     #

     # NOTE: special case for the plasecholType, this value
     must be passed as an array

     xml_string =

     "<holiday>

          <plasecholLength>0</plasecholLength>

          <plasecName>old Years Eve</plasecName>

          <plasecholType type='array'>

          <val>2</val>

          </plasecholType>

          </holiday>"


     req.body = xml_string


     # gotta give our login and password for authorization and
     authentication

     req.basic_auth(my_user, my_pwd)
```

```
    # set the content type to xml for REST commands
    req["Content-Type"] = "application/xml"

    #send the REST request
    resp = http.request(req)

    puts "RESPONSE: #{resp.header} \n\n"
    puts "DATA: "
    puts "#{resp.body}"

end
```

Output

```
RESPONSE: #<Net::HTTPCreated:0x007ffb209e2050>
```

Update

```
# holiday_update.rb
# sample REST call to update an existing holiday record
# <twl> Avigilon, Inc.
#
# arguments: ServerAddress, ServerPort, Username, password
#
my_server = ARGV[0]
my_port = ARGV[1]
my_user = ARGV[2]
my_pwd = ARGV[3]

require 'net/https'

http = Net::HTTP.new(my_server, my_port)
```

```
http.use_ssl = true

http.verify_mode = OpenSSL::SSL::VERIFY_NONE


http.start do |http|


     # The URL contains the CN of the holiday record we wish to
     update

     #

     # Special processing on Holidays, we must pass the
     plasecholDate value on the url rather than in the xml
     stream

     #

     req = Net::HTTP::Put.new
     ('/holidays/3d6129ce865f42cf.xml?plasecholDate=12/31/202
     1')

     # The body of the request contains xml of the attribute(s)
     to update

     #

     # NOTE: special case for the plasecholType, this value
     must be passed as an array

     xml_string =

     "<holiday>

          <plasecholLength>0</plasecholLength>

          <plasecName>NEW Years Eve</plasecName>

          <plasecholType type='array'>

          <val>3</val>

          <val>4</val>

          </plasecholType>

          </holiday>"


     req.body = xml_string


     # gotta give our login and password for authorization and
```

```
      authentication
      req.basic_auth(my_user, my_pwd)
      # set the content type to xml for REST commands
      req["Content-Type"] = "application/xml"
      #send the REST request
      resp = http.request(req)
      puts "RESPONSE: #{resp.header} \n\n"
      puts "DATA: "
      puts "#{resp.body}"
end
```

Identities

Identities are one of the primary objects in the ACM system; they store details about people and are the logical parent record for many details related to people. The structure of the ACM database is such that one Identity can have multiple Tokens and multiple Photos so manipulating Token and Photo objects is done separately from the Identity. In general you will first create an Identity and then create the Tokens and Photos for that Identity. Likewise to update a Token you must first know the unique identifier (CN) of the Identity. See the examples under Photos and Tokens for detailed information.

In addition to the pre-defined database fields for the Identity model users of the ACM system also have the ability to create User Defined Fields to extend the available attributes for an Identity.

Important: Searching for Identities requires that some search criteria be passed in. By default no identities will be returned if no search criteria is given. This is to prevent unintentionally overloading the server on systems with many identity records.

Important: To return user defined fields you must add “include_udf=true” as a parameter to the URL.

There are several special features available in the REST API to assist in managing identities including:

- The ability to perform advanced searches including: search by token number and search by modify date
- The ability to return user defined fields. To return User Defined fields add the parameter “?include_udf=true” to the URL.
- The ability to add and read JPEG data to for photos. Identity photos must be JPEG and are passed as base 64 encoded data.

Note: The Force Password Change field does not apply to identities managed through the REST API.

Model: identities

```
cn: CN

plasecLocation: Location

plasecName: Name

plasecPartitionDN: Partitions

plasecActivatedate: Activate Date

plasecAreaDN: Last Area

plasecAddress: Sequence Number

plasecDeactivatedate: Deactivate Date

plasecFname: First Name

plasecId: External System ID
```

plasecIdstatus: Status

plasecInternalnumber: Internal Number

plasecIssuedate: Issue Date

plasecLastdoor: Last Door

plasecLasttime: Last Used

plasecLname: Last Name

plasecLogin: Login

plasecLogindate: Login Date

plasecMI: Middle Name

plasecTyp: Type

plasecidentityAddress: Street Address

plasecidentityBuilding: Building

plasecidentityCity: City

plasecidentityCompany: Company

plasecidentityDefaultBadgeTmplDN: Default Badge Template

plasecidentityDefaultIdProfile: Default Identity Profile

plasecidentityDepartment: Department

plasecidentityDivision: Division

plasecidentityDoNotLogRESTLogins: Do Not Log REST Logins?

plasecidentityDomain: Remote Domain

plasecidentityEmailaddress: Email Address

plasecidentityExternalTrx: Allow Remote Access?

plasecidentityItemsperpage: Items/Page

plasecidentityLocale: Locale

plasecidentityMonitordfltrows: Monitor dflt rows

plasecidentityMaxActiveTokens: Maximum Active Token

plasecidentityPartitionsDN: Partitions

plasecidentityPhone: Phone

plasecidentityRemoteAuthenticate: Remote Authentication?

plasecidentityRoleDN: Role

plasecidentityShowtzoffset: Show Timezone Offset?

```
    plasecidentitySitelocation: Site Location
    plasecidentityState: State / Province
    plasecidentityTitle: Title
    plasecidentityWorkphone: Work Phone
    plasecidentityZipcode: Zip Code
```

Find All

The key point to note when searching for all identities is that by default no identities will be returned.

> **Important:** You must pass some search criteria in order to get any results.

```
# identities_all.rb
# sample REST call to get a list of identities
# <twl> Avigilon, Inc.
#
# arguments: ServerAddress, ServerPort, Username, password
#
my_server = ARGV[0]
my_port = ARGV[1]
my_user = ARGV[2]
my_pwd = ARGV[3]

require 'net/https'

http = Net::HTTP.new(my_server, my_port)
http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE

http.start do |http|
```

```
# this request is for a list of all identities

# NOTE: to prevent hanging the system with long identity queries

## this is a special case list. By default no identities will be
returned.

## to change this pass in a parameter "filter" with a different
search value i.e. plasecLname=*

## or any other valid ldap search filter

#

req = Net::HTTP::Get.new('/identities.xml?filter=plasecLname=*')

# add the incude_udf=true parameter if you'd like to get user
defined fields

# req = Net::HTTP::Get.new('/identities.xml?include_
udf=true&plasecLname=*')


# gotta give our login and password for authorization and
authentication

req.basic_auth(my_user, my_pwd)


# set the content type to xml for REST commands

req["Content-Type"] = "application/xml"


#send the REST request

resp = http.request(req)


puts "RESPONSE: #{resp.header} \n\n"

puts "DATA: "

puts "#{resp.body}"

puts "#{resp.header['location']}"


end
```

Response Examples

RESPONSE: `#<Net::HTTPOK:0x007fe484850438>`

DATA:

```
<?xml version="1.0" encoding="UTF-8"?>
<identities type="array">
     <identity>
          <dn>cn=0,ou=identities,dc=plasec</dn>
          <cns type="array">
               <cn>0</cn>
          </cns>
          <createTimestamp>20141011082444Z</createTimestamp>
          <creatorsName>cn=manager,dc=plasec</creatorsName>
          <entryDN>cn=0,ou=identities,dc=plasec</entryDN>
          <entryUUID>ccd1814c-e56b-1033-8ca6-
          e9a40486d665</entryUUID>
          <hasSubordinates>TRUE</hasSubordinates>
          <modifiersName>cn=0,ou=identities,dc=plasec</modifiersNam
          e>
          <modifyTimestamp>20160601163826Z</modifyTimestamp>
          <objectClasses type="array">
               <objectClass>plasecIdentity</objectClass>
          </objectClasses>
          <plasecActivatedate>20080702000000Z</plasecActivatedate>
          <plasecAddress>2</plasecAddress>
          <plasecCryptedpwd>
          {SSHA}RQgV/354QVAG1h9ISiGq9oCR3wJRVVI1</plasecCryptedpwd>
          <plasecDeactivatedate>20320702000000Z</plasecDeactivatedat
          e>
          <plasecFname>System</plasecFname>
          <plasecIdstatus>1</plasecIdstatus>
          <plasecIssuedate>20080702000000Z</plasecIssuedate>
          <plasecLname>Administrator</plasecLname>
          <plasecLogin>admin</plasecLogin>
          <plasecLogindate>20160601163826Z</plasecLogindate>
          <plasecName>Administrator, System</plasecName>
          <plasecUdefstatus>Active</plasecUdefstatus>
```

```
        <plasecidentityDoNotLogRESTLogins>FALSE</plasecidentityDoN
        otLogRESTLogins>

        <plasecidentityExternalTrx>FALSE</plasecidentityExternalTr
        x>

        <plasecidentityItemsperpage>500</plasecidentityItemsperpag
        e>

        <plasecidentityLocale>en-US</plasecidentityLocale>

        <plasecidentityMonitordfltrows>100</plasecidentityMonitord
        fltrows>

        <plasecidentityPagetimeout>604800000</plasecidentityPageti
        meout>

        <plasecidentityRemoteAuthenticate>FALSE</plasecidentityRem
        oteAuthenticate>

        <plasecidentityRoleDNs type="array">

             <plasecidentityRoleDN>cn=0,ou=roles,dc=plasec</p
             lasecidentityRoleDN>

        </plasecidentityRoleDNs>

        <plasecidentityShowtzoffset>FALSE</plasecidentityShowtzoff
        set>

        <structuralObjectClass>plasecIdentity</structuralObjectCla
        ss>

        <subschemaSubentry>cn=Subschema</subschemaSubentry>

        <userPasswords type="array">

             <userPassword>
             {SSHA}YJfni7gGDYiNqNT4z64Cglq47X1iSngu</userPass
             word>

        </userPasswords>

    </identity>

    <identity>

        <dn>cn=0539e15f79eb4c67,ou=identities,dc=plasec</dn>

        <cns type="array">

             <cn>0539e15f79eb4c67</cn>

        </cns>

        <createTimestamp>20150210093437Z</createTimestamp>

        <creatorsName>cn=0,ou=identities,dc=plasec</creatorsName>
```

```
<entryDN>cn=0539e15f79eb4c67,ou=identities,dc=plasec</entr
yDN>

<entryUUID>c61aaa5c-4553-1034-871f-
ad5662bd7c7d</entryUUID>

<hasSubordinates>TRUE</hasSubordinates>

<modifiersName>cn=manager,dc=plasec</modifiersName>

<modifyTimestamp>20150713221406Z</modifyTimestamp>

<objectClasses type="array">

     <objectClass>plasecIdentity</objectClass>

</objectClasses>

<plasecFname>Race</plasecFname>

<plasecId>111</plasecId>

<plasecIdstatus>1</plasecIdstatus>

<plasecLname>Bannon</plasecLname>

<plasecLogin>rbannon</plasecLogin>

<plasecLogindate>20150713221406Z</plasecLogindate>

<plasecMI>G</plasecMI>

<plasecName>Bannon, Race, G</plasecName>

<plasecSalt>75d505afdd443f70ac8688fbf4ef83ed940bf0b06e6183
2d7e8037e85f08fb1a</plasecSalt>

<plasecTyp>Contractor</plasecTyp>

<plasecgroupDNs type="array">

     <plasecgroupDN>cn=4d4a7febd5304bcd,ou=groups,dc=
     plasec</plasecgroupDN>

</plasecgroupDNs>

<plasecidentityDepartment>R&amp;amp;D</plasecidentityDepartmen
t>

<plasecidentityDivision>Commercial
Products</plasecidentityDivision>

<plasecidentityExternalTrx>FALSE</plasecidentityExternalTr
x>

<plasecidentityGroupsDNs type="array">
```

<plasecidentityGroupsDN>cn=4d4a7febd5304bcd,ou=groups,dc=plasec</plasecidentityGroupsDN>

</plasecidentityGroupsDNs>

<plasecidentityPagetimeout>600000</plasecidentityPagetimeout>

<plasecidentityPartitionsDNs type="array">

<plasecidentityPartitionsDN>cn=0c62d16db62e4dff,ou=partitions,dc=plasec</plasecidentityPartitionsDN>

</plasecidentityPartitionsDNs>

<plasecidentityRemoteAuthenticate>FALSE</plasecidentityRemoteAuthenticate>

<plasecidentityRoleDNs type="array">

<plasecidentityRoleDN>cn=1,ou=roles,dc=plasec</plasecidentityRoleDN>

<plasecidentityRoleDN>cn=9bf0f6d7e774444f,ou=roles,dc=plasec</plasecidentityRoleDN>

</plasecidentityRoleDNs>

<plasecidentityState>Alabama</plasecidentityState>

<plasecidentityTitle>Temporary</plasecidentityTitle>

<plasecidentityUdefsDNs type="array">

<plasecidentityUdefsDN>cn=0539e15f79eb4c67,cn=identities,cn=Birth Date,ou=udeffields,ou=lookups,dc=plasec</plasecidentityUdefsDN>

<plasecidentityUdefsDN>cn=0539e15f79eb4c67,cn=identities,cn=License Plate,ou=udeffields,ou=lookups,dc=plasec</plasecidentityUdefsDN>

<plasecidentityUdefsDN>cn=0539e15f79eb4c67,cn=identities,cn=SSN,ou=udeffields,ou=lookups,dc=plasec</plasecidentityUdefsDN>

<plasecidentityUdefsDN>cn=0539e15f79eb4c67,cn=identities,c

```
        n=int,ou=udeffields,ou=lookups,dc=plasec</plasecidentityUd
        efsDN>

             </plasecidentityUdefsDNs>

             <structuralObjectClass>plasecIdentity</structura
             lObjectClass>

             <subschemaSubentry>cn=Subschema</subschemaSubent
             ry>

             <userPasswords type="array">

             <userPassword>
             {SSHA}OFmSHrlt3TRJmLN0ACeQtd3BsMB6eUYv</userPass
             word>

        </userPasswords>

    </identity>

    .

    .

    .

    </identities>
```

Find One

```
# identity_find_one.rb

# sample REST call to get a single Identity record

# <twl> Avigilon, Inc.

#

# arguments: ServerAddress, ServerPort, Username, password

#

my_server = ARGV[0]

my_port = ARGV[1]

my_user = ARGV[2]

my_pwd = ARGV[3]

my_id_cn = ARGV[4]


require 'net/https'
```

```
http = Net::HTTP.new(my_server, my_port)

http.use_ssl = true

http.verify_mode = OpenSSL::SSL::VERIFY_NONE


http.start do |http|


# The identity common name (CN) for the desired identity must be passed as
part of the url

# you get the cn from the identities query. It's <CN> or also is part of the
full Distinguished Name (DN)

# <dn>cn=8a5692efcaf14b1d,ou=identities,dc=plasec</dn>

# <cn>8a5692efcaf14b1d</cn>

#


# if we want user defined fields then add the include_udf=true parameter
(NOTE: UDF's are a feature new in 5.6)

my_url = "/identities/" + my_id_cn + ".xml?include_udf=true"

req = Net::HTTP::Get.new(my_url)


# gotta give our login and password for authorization and authentication

req.basic_auth(my_user, my_pwd)


# set the content type to xml for REST commands

req["Content-Type"] = "application/xml"


#send the REST request

resp = http.request(req)


puts "RESPONSE: #{resp.header} \n\n"

puts "DATA: "

puts "#{resp.body}"


end
```

Response Example

RESPONSE: #<Net::HTTPOK:0x007fb41401c618>

DATA:

```
<identity>
     <dn>cn=0539e15f79eb4c67,ou=identities,dc=plasec</dn>
     <cns type="array">
          <cn>0539e15f79eb4c67</cn>
     </cns>
     <createTimestamp>20150210093437Z</createTimestamp>
     <creatorsName>cn=0,ou=identities,dc=plasec</creatorsName>
     <entryDN>cn=0539e15f79eb4c67,ou=identities,dc=plasec</entryDN>
     <entryUUID>c61aaa5c-4553-1034-871f-ad5662bd7c7d</entryUUID>
     <hasSubordinates>TRUE</hasSubordinates>
     <modifiersName>cn=manager,dc=plasec</modifiersName>
     <modifyTimestamp>20150713221406Z</modifyTimestamp>
     <objectClasses type="array">
          <objectClass>plasecIdentity</objectClass>
     </objectClasses>
     <plasecFname>Race</plasecFname>
     <plasecId>111</plasecId>
     <plasecIdstatus>1</plasecIdstatus>
     <plasecLname>Bannon</plasecLname>
     <plasecLogin>rbannon</plasecLogin>
     <plasecLogindate>20150713221406Z</plasecLogindate>
     <plasecMI>G</plasecMI>
     <plasecName>Bannon, Race, G</plasecName>
     <
     plasecSalt>75d505afdd443f70ac8688fbf4ef83ed940bf0b06e61832d7e8037e85
     f08fb1a</plasecSalt>
     <plasecTyp>Contractor</plasecTyp>
     <plasecgroupDNs type="array">
```

```
     <plasecgroupDN>cn=4d4a7febd5304bcd,ou=groups,dc=plasec</pl
     asecgroupDN>

</plasecgroupDNs>

<plasecidentityDepartment>R&amp;amp;D</plasecidentityDepartment>

<plasecidentityDivision>Commercial Products</plasecidentityDivision>

<plasecidentityExternalTrx>FALSE</plasecidentityExternalTrx>

<plasecidentityGroupsDNs type="array">

     <plasecidentityGroupsDN>cn=4d4a7febd5304bcd,ou=groups,dc=p
     lasec</plasecidentityGroupsDN>

</plasecidentityGroupsDNs>

<plasecidentityPagetimeout>600000</plasecidentityPagetimeout>

<plasecidentityPartitionsDNs type="array">

     <plasecidentityPartitionsDN>cn=0c62d16db62e4dff,ou=partiti
     ons,dc=plasec</plasecidentityPartitionsDN>

</plasecidentityPartitionsDNs>

<plasecidentityRemoteAuthenticate>FALSE</plasecidentityRemoteAuthent
icate>

<plasecidentityRoleDNs type="array">

     <plasecidentityRoleDN>cn=1,ou=roles,dc=plasec</plasecident
     ityRoleDN>

     <plasecidentityRoleDN>cn=9bf0f6d7e774444f,ou=roles,dc=plas
     ec</plasecidentityRoleDN>

</plasecidentityRoleDNs>

<plasecidentityState>Alabama</plasecidentityState>

<plasecidentityTitle>Temporary</plasecidentityTitle>

<plasecidentityUdefsDNs type="array">

     <plasecidentityUdefsDN>cn=0539e15f79eb4c67,cn=identities,c
     n=Birth
     Date,ou=udeffields,ou=lookups,dc=plasec</plasecidentityUde
     fsDN>

      <plasecidentityUdefsDN>cn=0539e15f79eb4c67,cn=identities,cn=Li
cense
Plate,ou=udeffields,ou=lookups,dc=plasec</plasecidentityUdefsDN>

<plasecidentityUdefsDN>cn=0539e15f79eb4c67,cn=identities,cn=SSN,ou=u
deffields,ou=lookups,dc=plasec</plasecidentityUdefsDN>
```

```
      <plasecidentityUdefsDN>cn=0539e15f79eb4c67,cn=identities,cn=int,ou=u
      deffields,ou=lookups,dc=plasec</plasecidentityUdefsDN>


           </plasecidentityUdefsDNs>

           <structuralObjectClass>plasecIdentity</structuralObjectCla
           ss>

           <subschemaSubentry>cn=Subschema</subschemaSubentry>

           <userPasswords type="array">

                <userPassword>
                {SSHA}OFmSHrlt3TRJmLN0ACeQtd3BsMB6eUYv</userPass
                word>

           </userPasswords>

           <userDefined>

           <Birth-Date type="dateTime">1950-04-01T00:00:00Z</Birth-
           Date>

           <License-Plate>L120-SFD</License-Plate>

           <SSN>123-12-1233</SSN>

           <int type="integer">100</int>

      </userDefined>
</identity>
```

Advanced Search

```
# identity_advanced_search.rb

# sample REST call to perform an advanced search on the identity model

# <twl> Avigilon, Inc.

#

# arguments: ServerAddress, ServerPort, Username, password

#

my_server = ARGV[0]

my_port = ARGV[1]

my_user = ARGV[2]

my_pwd = ARGV[3]


require 'net/https'
```

```
http = Net::HTTP.new(my_server, my_port)

http.use_ssl = true

http.verify_mode = OpenSSL::SSL::VERIFY_NONE


http.start do |http|


    # The following extra parameters perform an advanced search on the
    identity model.

    # adv_search_exec_search=true - This tells the index action to
    perform the advanced search

    #

    # The following two parameters contain the search parameters to be
    used in the search the training number "0" is incremented to add
    additional attributes to the search

    # adv_search_field_0=plasecInternalnumber - This parameter along
    with 'adv_search_valX' pass in attribute/value pairs to be used in
    the query

    # adv_search_field_0 should have a corresponding search_pattern_0
    and adv_search_field_1 should have a corresponding search_pattern_1

    # search_pattern values are:

    ## 1: Equals

    ## 2: Begins With

    ## 3: Contains

    ## 4: Ends With

    ## 5: Not Equals

    ## 6: Is Empty

    ## 7: Greater than (Token Internal Number or Maximum Active Token)

    ## Example: adv_search_field_0=plasecInternalnumber&adv_search_val_
    0=1000&search_pattern_0=7

    ## 8: Less than (Token Internal Number or Maximum Active Token)

    ## Example: adv_search_field_0=plasecidentityMaxActiveTokens&adv_
    search_val_0=500&search_pattern_0=8

    ## &gt;=: After (Applies to plasecActivatedate,
    plasecDeactivatedate, Token_plasecIssuedate, date, modifyTimestamp,
```

```
or createTimestamp)

## Example: adv_search_field_0=plasecActivatedate&adv_search_val_
0=01/01/2020&search_pattern_0=%3E

## &lt;=: Before (Applies to plasecActivatedate,
plasecDeactivatedate, Token_plasecIssuedate, date, modifyTimestamp,
createTimestamp)

## Example: adv_search_field_0=plasecActivatedate&adv_search_val_
0=01/01/2020&search_pattern_0=%3C

#

# adv_search_val_0=101 - This parameter hold the value for the
field0 attribute to be used in the search

#

# Some common attributes that can be searched include: (Description
- attributeName) use the attributeName

## Last Name - "plasecLname"

## Division - "plasecidentityDivision"

## Issue Date - "plasecIssuedate"

## External Sys ID - "plasecId"

## Department - "plasecidentityDepartment"

## First Name - "plasecFname"

## Token Internal # - "plasecInternalnumber"

## Token Embossed # - "plasecEmbossednumber"

## Token Activate Date - "plasecActivatedate"

## Token Deactivate Date - "plasecDeactivatedate"

## Token Status - "plasecTokenstatus"


# This next parameter indicates whether the search should use a
logical 'and' (&) or a logical 'or' (|)

# adv_search_and_or=[&,|](%26,%7C)


# For REST calls this next parameter should be set to the number of search
criteria

# ** note that the count actually starts with '0' for the first one.

# adv_search_cnt=0
```

```
#

# Below are several sample URL's. Uncomment the url for the type of
search you wish to perform.

#


#

# sample search using the internal number (=111)

#

url = '/identities.xml?adv_search_exec_search=true&adv_search_field_
0=plasecId&adv_search_val_0=111&adv_search_and_or=&adv_search_cnt=0'


#

## sample search using the Token Internal # - "plasecInternalnumber"

#

#url = '/identities.xml?adv_search_exec_search=true&adv_search_
field_0=plasecInternalnumber&adv_search_val_0=543&adv_search_and_
or=&adv_search_cnt=0'


#

## sample search using multiple attributes (Last name starts with
"A" or "B")

#

#url = "/identities.xml?adv_search_exec_search=true&adv_search_
field_0=plasecLname&adv_search_val_0=B&search_pattern_0=2&adv_
search_field_1=plasecLname&adv_search_val_1=A&search_pattern_
1=2&adv_search_cnt=1&adv_search_and_or=%7C"


#

## sample search using multiple attributes (Last name starts with
"A" and "B" - of course should be no records)

#

#url = '/identities.xml?adv_search_exec_search=true&adv_search_
field_0=plasecLname&adv_search_val_0=B&search_pattern_0=2&adv_
search_field_1=plasecLname&adv_search_val_1=A&search_pattern_
1=2&adv_search_cnt=1&adv_search_and_or=%26'
```

```
    #
    ## sample search using filter parameter
    #
    #url = '/identities.xml?filter=(plasecLname=A*)'


    #
    ## sample search using NOT criteria
    #
    #url = '/identities.xml?filter=!(plasecidentityDivision=Comme*)'
    #url = '/identities.xml?filter=!(plasecId=EasyLobby_Visitor*)'


    # this request is for a list of identities that meet the criteria
    req = Net::HTTP::Get.new(url)
    #req = Net::HTTP::Get.new('/identities.xml?filter=
    (plasecId=EasyLobby_Visitor*)')


    req.content_type = 'content/xml'


    # gotta give our login and password for authorization and
    authentication
    req.basic_auth(my_user, my_pwd)


    # set the content type to xml for REST commands
    req["Content-Type"] = "application/xml"


    #send the REST request
    resp = http.request(req)


    puts "RESPONSE: #{resp.header} \n\n"
    puts "DATA: "
    puts "#{resp.body}"
    puts "#{resp.header['location']}"
end
```

Find by External System ID

Use the following url to find an identity by the External System ID:

```
url = '/identities.xml?adv_search_exec_search=true&adv_search_field_
0=plasecId&adv_search_val_0=111&adv_search_and_or=&adv_search_cnt=0'
```

Find by Multiple Criteria

To search by a combination of attributes add more search_field/val pairs and the adv_search_and_or parameter as shown below. This search will find all identities where the last name begins with A or B.

```
url = "/identities.xml?adv_search_exec_search=true&adv_search_field_
0=plasecLname&adv_search_val_0=B&search_pattern_0=2&adv_search_field_
1=plasecLname&adv_search_val_1=A&search_pattern_0=2&&adv_search_cnt=1&adv_
search_and_or=%7C"
```

Find by Token Number

To find an identity record based upon the Token Internal number you can use a URL similar to this.

```
url = '/identities.xml?adv_search_exec_search=true&adv_search_field_
0=plasecInternalnumber&adv_search_val_0=111&adv_search_and_or=&adv_search_
cnt=0'
```

Advanced Filter Searches

There is also an option to write your own ldap filter strings. You can pass in an ldap filter using the ‘filter’ parameter on the URL and add a valid ldap query filter. This expands the search capability to basically any valid ldap search.

For example we show a search below querying for all identities who’s last name begins with the letter ‘A’.

```
url = '/identities.xml?filter=(plasecLname=A*)'
```

The ‘filter’ parameter can be useful for performing “NOT” searches as shown below.

```
req = Net::HTTP::Get.new('/identities.xml?filter=!(plasecId=EasyLobby_
Visitor*)')
```

Search by Modify Date

Find Identities by Role

Create

```
# identity_create.rb
# sample REST call to create a new identity
# <twl> Avigilon, Inc.
#
# arguments: ServerAddress, ServerPort, Username, password
#
my_server = ARGV[0]
my_port = ARGV[1]
my_user = ARGV[2]
my_pwd = ARGV[3]
my_lname = ARGV[4]
my_fname = ARGV[5]

require 'net/https'

http = Net::HTTP.new(my_server, my_port)
http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE

http.start do |http|

     # this request is to create an identity
     req = Net::HTTP::Post.new('/identities.xml')
     # The body of the request contains xml of the attribute(s) to update
     # REQUIRED FIELDS INCLUDE: plasecIdstatus (1=active), plasecLname
     #
```

```
    xml_string =
    "<identity>
         <plasecIdstatus>1</plasecIdstatus>
         <plasecLname>#{my_lname}</plasecLname>
         <plasecFname>#{my_fname}</plasecFname>
    </identity>"
    req.body = xml_string

    # gotta give our login and password for authorization and
    authentication
    req.basic_auth(my_user, my_pwd)

    # set the content type to xml for REST commands
    req["Content-Type"] = "application/xml"

    #send the REST request
    resp = http.request(req)

    puts "RESPONSE: #{resp.header} \n\n"
    puts "DATA: "
    puts "#{resp.body}"
    puts "#{resp.header['location']}"

end
```

Update Roles

```
# identity_role_update.rb
# sample REST call to update the Roles for an identity
# <twl> Avigilon, Inc.
#
# arguments: ServerAddress, ServerPort, Username, password
#
my_server = ARGV[0]
my_port = ARGV[1]
my_user = ARGV[2]
```

```
my_pwd = ARGV[3]

require 'net/https'

http = Net::HTTP.new(my_server, my_port)
http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE

http.start do |http|

  # this request is to update roles on an identity
  # The role DN's must be passed in the 'sel_members' attribute as an array
  #   as show below
  #
  #
  #  Pass the Identity CN on the URL
  #
  req = Net::HTTP::Put.new('/identities/b48498bb5b904a90/update_roles.xml')

  # The body of the request contains xml of the attribute(s) to update
  # 'sel_members' contains all the roles that should be on this identity
  xml_string = '<sel_members type="array">
      <sel_member>cn=2,ou=roles,dc=plasec</sel_member>
      <sel_member>cn=0,ou=roles,dc=plasec</sel_member>
    </sel_members>'

  req.body = xml_string

  # gotta give our login and password for authorization and authentication
  req.basic_auth(my_user, my_pwd)

  # set the content type to xml for REST commands
  req["Content-Type"] = "application/xml"

  #send the REST request
  resp = http.request(req)

  puts "RESPONSE: #{resp.header} \n\n"
  puts "DATA: "
  puts "#{resp.body}"

end
```

Update

```
# identity_update.rb
# sample REST call to get a update an identity
# <twl> Avigilon, Inc.
#
```

```
# arguments: ServerAddress, ServerPort, Username, password
#
my_server = ARGV[0]
my_port = ARGV[1]
my_user = ARGV[2]
my_pwd = ARGV[3]

require 'net/https'

http = Net::HTTP.new(my_server, my_port)
http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE

http.start do |htt/Work/shared/api_samples/Doc p|

  # this request is to update an identity
  # pass the CN of the identity to be updated on the URL
  # and the values to update in the body, in this case we're updating External
System ID (plasecId)
  #
  req = Net::HTTP::Put.new('/identities/b48498bb5b904a90.xml')

  # The body of the request contains xml of the attribute(s) to update
  xml_string =
  '<identity>
    <plasecId>54321</plasecId>
  </identity>'

  req.body = xml_string

  # gotta give our login and password for authorization and authentication
  req.basic_auth(my_user, my_pwd)

  # set the content type to xml for REST commands
  req["Content-Type"] = "application/xml"

  #send the REST request
  resp = http.request(req)

  puts "RESPONSE: #{resp.header} \n\n"
  puts "DATA: "
  puts "#{resp.body}"
  puts "#{resp.header['location']}"

end
```

User Defined Fields

User defined fields are custom fields that you can add to the Identities page in ACM to capture organization

information for each identity.

Update

```
# udef_field_update.rb
# sample REST call to update a user defined field
# <twl> Avigilon, Inc.
#
# arguments: ServerAddress, ServerPort, Username, password
#
my_server = ARGV[0]
my_port = ARGV[1]
my_user = ARGV[2]
my_pwd = ARGV[3]
require 'net/https'

http = Net::HTTP.new(my_server, my_port)
http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE

http.start do |http|

  # this request is to update the UDF fields on an identity
  #req = Net::HTTP::Put.new('/identities/f23d533731d04340.xml?UDF_
Birth+Date=04/01/1990')
  #req = Net::HTTP::Put.new('/identities/f23d533731d04340.xml?UDF_
License+Plate=DILLIGAF')
  #req = Net::HTTP::Put.new('/identities/f23d533731d04340.xml?UDF_
License+Plate=DILLIGAF&UDF_Birth+Date=04/01/1990')
  #req = Net::HTTP::Put.new('/identities/f23d533731d04340.xml?UDF_
License+Plate=DILLIGAF&UDF_Birth+Date=04/01/1990')
  #req = Net::HTTP::Put.new('/identities/f23d533731d04340.xml?UDF_
License+Plate=')
  #req = Net::HTTP::Put.new('/identities/b9ad29e6aba04f39.xml?UDF_
License+Plate=1BUD&UDF_Birth+Date=04/01/1990')
  #req = Net::HTTP::Put.new('/identities/0539e15f79eb4c67.xml?UDF_
License+Plate=2BUDs')
  #req = Net::HTTP::Put.new('/identities/0539e15f79eb4c67.xml?UDF_
License+Plate=3BUDs&UDF_Birth+Date=04/01/1980')
  #req = Net::HTTP::Put.new('/identities/b48498bb5b904a90.xml?UDF_
License+Plate=3BUDs&UDF_Birth+Date=04/01/1980')
  #req = Net::HTTP::Put.new('/identities/0539e15f79eb4c67.xml?UDF_
License+Plate=4BUDs&UDF_Location=test')
  #req = Net::HTTP::Put.new('/identities/0539e15f79eb4c67.xml?UDF_
Birth+Date=04/01/1991')
  #req = Net::HTTP::Put.new('/identities/b9ad29e6aba04f39.xml?UDF_
Birth+Date=04/01/1990')
  #req = Net::HTTP::Put.new('/identities/b9ad29e6aba04f39.xml?UDF_
License+Plate=&UDF_Birth+Date=')
  req = Net::HTTP::Put.new('/identities/b48498bb5b904a90.xml?UDF_
```

```
License+Plate=3BUDs4u&UDF_EA+Credits=980')

  # The body of the request contains xml of the attribute(s) to update
  # but blank for user defined field updates
  xml_string = ''

  req.body = xml_string

  # gotta give our login and password for authorization and authentication
  req.basic_auth(my_user, my_pwd)

  #send the REST request
  resp = http.request(req)

  puts "RESPONSE: #{resp.header} \n\n"
  puts "DATA: "
  puts "#{resp.body}"
  puts "#{resp.header}"

end
```

Get Update

```
# identity_udef_update.rb
# sample REST call to get a update User Defined Fields for an Identity
# <twl> Avigilon, Inc.
#
# arguments: ServerAddress, ServerPort, Username, password
#
my_server = ARGV[0]
my_port = ARGV[1]
my_user = ARGV[2]
my_pwd = ARGV[3]

require 'net/https'

http = Net::HTTP.new(my_server, my_port)
http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE

http.start do |http|

  # this request is to update the User Defined Fields on an identity record
  # The CN for the identity to be udpated is passed on the URL
  # The values to be updated must be passed on the URL as shown here:
  #
  #req = Net::HTTP::Put.new('/identities/e104f1dff27446f2.xml?UDF_SSN=123-12-
1234&UDF_int=999123')
  #req = Net::HTTP::Put.new('/identities/e104f1dff27446f2.xml?UDF_Word=Bird')
```

```
  #req = Net::HTTP::Put.new('/identities/d52718e1cf434cb1.xml?UDF_Reports-
to=Bird%20Man')
  #req = Net::HTTP::Put.new('/identities/b48498bb5b904a90.xml?UDF_EA_
Credits=17&UDF_License_Plate=2FD321')
  #req = Net::HTTP::Put.new('/identities/b48498bb5b904a90.xml?UDF_EA_
Credits=17')
  #req = Net::HTTP::Put.new('/identities/b48498bb5b904a90.xml?UDF_Integer_
One=7&UDF_String_1=abcdefg')
  #req = Net::HTTP::Put.new('/identities/b48498bb5b904a90.xml?UDF_Integer_
One=17')

  xml_string = ''

  req.body = xml_string

  # gotta give our login and password for authorization and authentication
  req.basic_auth(my_user, my_pwd)

  # set the content type to xml for REST commands
  req["Content-Type"] = "application/xml"

  #send the REST request
  resp = http.request(req)

  puts "RESPONSE: #{resp.header} \n\n"
  puts "DATA: "
  puts "#{resp.body}"
  puts "#{resp.header['location']}"

end
```

Photo

Create

> **Note:** Supported in ACM API version 1.01 or later.

```
# identity_photo_create.rb

# sample REST call to add a photo to an Identity

# <twl> Avigilon, Inc.

#

# arguments: ServerAddress, ServerPort, Username, password

#
```

```
my_server = ARGV[0]
my_port = ARGV[1]
my_user = ARGV[2]
my_pwd = ARGV[3]

require 'net/https'
require 'base64'

http = Net::HTTP.new(my_server, my_port)
http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE

# grab a jpeg image from the file system
fid1 = File.open('/Work/Shared/Photos/Photos/jquest.jpeg', 'r')
# base 64 encode the image
pic1 = Base64.encode64(fid1.read)

http.start do |http|

# this request is to add a photo to an identity
# The CN of the identity is used as part of the URL
#
req = Net::HTTP::Post.new('/identities/0539e15f79eb4c67/photos.xml')

# The body of the request contains xml of the attribute(s) to update
xml_string =
"<photo>
     <plasecidentityPrimaryimage>FALSE</plasecidentityPrimaryimage>
     <plasecidentityContentype>image/jpeg</plasecidentityContentype>
     <plasecidentityFilename>testfid1.jpeg</plasecidentityFilename>
     <encoded_image>#{pic1}</encoded_image>
</photo>"
```

```
req.body = xml_string

# gotta give our login and password for authorization and authentication
req.basic_auth(my_user, my_pwd)

# set the content type to xml for REST commands
req["Content-Type"] = "application/xml"

#send the REST request
resp = http.request(req)

puts "RESPONSE: #{resp.header} \n\n"
puts "DATA: "
puts "#{resp.body}"

end
```

Read

```
# identity_photo_list.rb
# sample REST call to get all the photos for an identity
# <twl> Avigilon, Inc.
#
# arguments: ServerAddress, ServerPort, Username, password
#
my_server = ARGV[0]
my_port = ARGV[1]
my_user = ARGV[2]
my_pwd = ARGV[3]

require 'net/https'
require 'net/https'
```

```
require 'rubygems'
require 'xmlsimple'
require 'base64'

http = Net::HTTP.new(my_server, my_port)
http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE

http.start do |http|

     # this request is for a list of the photos for this identity
     req = Net::HTTP::Get.new('/identities/0539e15f79eb4c67/photos.xml')

     req.content_type = 'text/xml'

     # gotta give our login and password for authorization and
     authentication
     req.basic_auth(my_user, my_pwd)

     # set the content type to xml for REST commands
     req["Content-Type"] = "application/xml"

     #send the REST request
     @resp = http.request(req)

     puts "RESPONSE: #{@resp.header} \n\n"
     puts "DATA: "
     puts "#{@resp.body}"

end

#
```

```
# the body comes back as xml with the photos base 64 encoded
#
#
# For this sample we'l just grab the first photo object, decode it and
# save it to a file as a jpeg image
#
x1 = XmlSimple.xml_in(@resp.body)

# create a temp file to write the jpeg
fid = File.new('/tmp/tpic1.jpeg', 'w')
fid.puts(Base64.decode64(x1['object'][0]['binaryPhoto'][0]))
fid.close
```

Update

```
# idenity_photo_update.rb
# sample REST call to update an identity photo
# <twl> Avigilon, Inc.
#
# arguments: ServerAddress, ServerPort, Username, password
#
my_server = ARGV[0]
my_port = ARGV[1]
my_user = ARGV[2]
my_pwd = ARGV[3]

require 'net/https'
require 'base64'

http = Net::HTTP.new(my_server, my_port)
http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE
```

```
http.start do |http|

      # this request is to update the photo for an identity

      req = Net::HTTP::Put.new
      ('/identities/0539e15f79eb4c67/photos/0.xml')

      # The body of the request contains xml of the attribute(s) to update

      xml_string =

      "<photo>

      <plasecidentityPrimaryimage>TRUE</plasecidentityPrimaryimage>

      </photo>"


      req.body = xml_string


      # gotta give our login and password for authorization and
      authentication

      req.basic_auth(my_user, my_pwd)


      # set the content type to xml for REST commands

      req["Content-Type"] = "application/xml"


      #send the REST request

      resp = http.request(req)


      puts "RESPONSE: #{resp.header} \n\n"

      puts "DATA: "

      puts "#{resp.body}"


end
```

Delete

```
# identity_photo_delete.rb

# sample REST call to delete a photo record from an identity

# <twl> Avigilon, Inc.
```

```
#
# arguments: ServerAddress, ServerPort, Username, password
#
my_server = ARGV[0]
my_port = ARGV[1]
my_user = ARGV[2]
my_pwd = ARGV[3]

require 'net/https'

http = Net::HTTP.new(my_server, my_port)
http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE

http.start do |http|

    # this request is to delete one specific photo for this identity -
    CN==7
    # we need to know the CN of the identity and the CN of the photo to
    be deleted
    # the url is /identities/<identity CN>/photos/<Photo CN>.xml
    #
    req = Net::HTTP::Delete.new
    ('/identities/0539e15f79eb4c67/photos/1.xml')

    # gotta give our login and password for authorization and
    authentication
    req.basic_auth(my_user, my_pwd)

    # set the content type to xml for REST commands
    req["Content-Type"] = "application/xml"

    #send the REST request
```

```
    resp = http.request(req)

    puts "RESPONSE: #{resp.header} \n\n"
    puts "DATA: "
    puts "#{resp.body}"

end
```

Inputs

Inputs are associated with panels or doors and can include:

- Motion sensors
- Door contacts
- Smoke detectors
- REX (request to exit) buttons
- Perimeter and fence alarms
- Break glass window sensors
- Crash bars
- Capacitance duct sensors
- Device tamper switches

Model: inputs

```
cn: CN
description: Description
plasecAddress: Address
plasecAltname: Alt Name
plasecId: ID
plasecInstalled: Installed
plasecLocation: Location
plasecName: Name
plasecPartitionDN: Partitions
plasecCameraDN: Camera
plasecRCPSupervisionDN: Supervision
plasecTimezoneDN: Schedule
plasecinputDebounce: Debounce
plasecinputEntrydelay: Entry Delay
plasecinputExitdelay: Exit Delay
plasecinputHoldtime: Hold Time
plasecinputIprlevelDN: EOL resistance
plasecinputLogging: Logging
```

```
plasecinputMask: Masked

plasecinputMasklinkage: Mask Linkage

plasecinputMasktrouble: Mask trouble

plasecinputMode: Mode

plasecinputThreshold: Threshold

plasecidentityGroupsDN: Group
```

Example Responses

All Inputs with Partitioning

```
<inputs type="array">

<input>

<dn>cn=100,cn=0,cn=1,ou=panels,cn=1469e808914a45b1,ou=gateways,dc=pl
asec</dn>

<plasecPartitionDN type="array">

<plasecPartitionDN>cn=04190bfaba8141c1,ou=partitions,dc=plasec</plas
ecPartitionDN>

</plasecPartitionDN>

<plasecName>Input on subpanel 0 Address 1</plasecName>

<entryUUID>51f65672-3b00-103b-80cb-cfdd4f8dde2d</entryUUID>

<plasecInstalled>TRUE</plasecInstalled>

<isInUse>FALSE</isInUse>

</input>

<input>

<dn>cn=101,cn=0,cn=1,ou=panels,cn=1469e808914a45b1,ou=gateways,dc=pl
asec</dn>

<plasecName>Input on subpanel 0 Address 2</plasecName>

<entryUUID>52021b4c-3b00-103b-80cd-cfdd4f8dde2d</entryUUID>

<plasecInstalled>TRUE</plasecInstalled>

<isInUse>FALSE</isInUse>

</input>

</inputs>
```

All Inputs for a Single Subpanel


```
<inputs type="array">

<input>

<dn>cn=100,cn=0,cn=1,ou=panels,cn=1469e808914a45b1,ou=gateways,dc=pl
asec</dn>

<cns type="array">

<cn>100</cn>

</cns>

<entryUUID>51f65672-3b00-103b-80cb-cfdd4f8dde2d</entryUUID>

<objectClasses type="array">

<objectClass>plasecInput</objectClass>

</objectClasses>

<plasecAddress>1</plasecAddress>

<plasecInstalled>TRUE</plasecInstalled>

<plasecIsDefaultName>TRUE</plasecIsDefaultName>

<plasecName>Input on subpanel 0 Address 1</plasecName>

<plasecPartitionDNs type="array">

<plasecPartitionDN>cn=04190bfaba8141c1,ou=partitions,dc=plasec</plas
ecPartitionDN>

</plasecPartitionDNs>

<plasecRCPSupervisionDN>cn=8,ou=RcpSupervision,dc=plasec</plasecRCPS
upervisionDN>

<plasecVersion>1</plasecVersion>

<plasecinputDebounce>0</plasecinputDebounce>

<plasecinputEntrydelay>0</plasecinputEntrydelay>

<plasecinputExitdelay>0</plasecinputExitdelay>

<plasecinputHoldtime>0</plasecinputHoldtime>

<plasecinputLogging>1</plasecinputLogging>

<plasecinputMask>FALSE</plasecinputMask>

<plasecinputMasklinkage>FALSE</plasecinputMasklinkage>

<plasecinputMasktrouble>FALSE</plasecinputMasktrouble>

<plasecinputMode>0</plasecinputMode>
```

```
</input>
<input>
<dn>cn=101,cn=0,cn=1,ou=panels,cn=1469e808914a45b1,ou=gateways,dc=pl
asec</dn>
<cns type="array">
<cn>101</cn>
</cns>
<entryUUID>52021b4c-3b00-103b-80cd-cfdd4f8dde2d</entryUUID>
<objectClasses type="array">
<objectClass>plasecInput</objectClass>
</objectClasses>
<plasecAddress>2</plasecAddress>
<plasecInstalled>TRUE</plasecInstalled>
<plasecIsDefaultName>TRUE</plasecIsDefaultName>
<plasecName>Input on subpanel 0 Address 2</plasecName>
<plasecRCPSupervisionDN>cn=8,ou=RcpSupervision,dc=plasec</plasecRCPS
upervisionDN>
<plasecVersion>1</plasecVersion>
<plasecinputDebounce>0</plasecinputDebounce>
<plasecinputEntrydelay>0</plasecinputEntrydelay>
<plasecinputExitdelay>0</plasecinputExitdelay>
<plasecinputHoldtime>0</plasecinputHoldtime>
<plasecinputLogging>1</plasecinputLogging>
<plasecinputMask>FALSE</plasecinputMask>
<plasecinputMasklinkage>FALSE</plasecinputMasklinkage>
<plasecinputMasktrouble>FALSE</plasecinputMasktrouble>
<plasecinputMode>0</plasecinputMode>
</input>
</inputs>
```

Get Installed or Uninstalled Inputs

Note: The following requests and responses are supported in API version 1.11 or later.

Request Examples

Get all the installed and in-use inputs for the specified attributes.

XML

```
https://<my_ACM_Server>/rest/inputs.xml?attributes=plasecName,dn,isInUse,
plasecPartitionDN,entryUUID,plasecInstalled&installed=true&inuse=false
```

JSON

```
https://<my_ACM_Server>/rest/inputs.json?attributes=plasecName,dn,isInUse,
plasecPartitionDN,entryUUID,plasecInstalled&installed=true&inuse=false
```

Intrusion Panels

Note: The following requests and responses are supported in API version 1.12 or later.

Use the Bosch intrusion panels to monitor their intrusion areas, points, outputs and users in the ACM system.

Use the Digital Monitoring Products (DMP) intrusion panels to monitor their intrusion areas, zones, outputs and users in the ACM system.

Model

Field	Format	Definition
cn	String	The common name.
gateway_dn	String	The distinguished name. The unique identifier of configuration data.
plasecName	String	The name of the panel.
plasecExternalSystemAddress	String	The IP address or hostname of the panel.
plasecExternalSystemPort	String	The TCP/IP port number configured on the panel.
plasecExternalSystemPassword	6-24 character string	The Automation Passcode in Bosch Remote Programming Software, or the Remote Key in Remote Link™ software (no spaces).
plasecInstalled	String	TRUE or FALSE
plasecPartitionDN	String	The distinguished names of the partitions defined in the panel configuration.
plasecExternalSystemAltPassword	String	Bosch intrusion system only. The Remote App Passcode in the Bosch Remote Programming Software.
plasecExternalSystemAccount	String	DMP intrusion system only. The account number in the Remote Link software.

Note: Zones, points, areas, and outputs are edited in intrusion panel manufacturer software (they are not edited in the ACM system).

Output

Field	Format	Definition
Return code	String	The ACM system response: 200 — Success 404 — Access denied

List Intrusion Panels

Request Examples

Get a list of the DMP and Bosch intrusion panels.

XML

```
https://<my_ACM_Server>/dmp_intrusions.xml
```

JSON

```
https://<my_ACM_Server>/bosch_intrusions.json
```

Update Intrusion Panel

Request Example

Update a DMP intrusion panel with the specified attributes.

XML

PUT

```
https://<my_ACM_Server>/dmp_intrusions/cn=2,ou=externalsystems,
cn=5d724c5a5c2449af,ou=gateways,dc=plasec.xml "<dmp-
intrusion><plasecName>NameUpdated</plasecName></dmp-intrusion>"
```

Note: Only the XML-based request is supported.

Delete Intrusion Panel

Note: Associated items such as global linkages and actions, zones, areas, outputs, identity mapping, and intrusion users will also be deleted. The recommended best practice is to use the plasecInstalled flag to uninstall an object instead of deleting it.

Request Example

Delete an intrusion panel with the specified attributes.

XML

DELETE

```
https://<my_ACM_Server>/dmp_intrusions/cn=2,ou=externalsystems,cn=
5d724c5a5c2449af,ou=gateways,dc=plasec.xml
```

Note: Only the XML-based request is supported.

Map Templates

Model: map_tmpls

```
cn: CN
description: Description
plasecAddress: Address
plasecAltname: Alt Name
plasecId: ID
plasecInstalled: Installed
plasecLocation: Location
plasecName: Name
plasecmaptmplFilename: File
plasecmaptmplWidth: Width
plasecmaptmplHeight: Height
```

Outputs

Model: outputs

```
cn: CN
description: Description
plasecAddress: Address
plasecAltname: Alt Name
plasecId: ID
plasecInstalled: Installed
plasecLocation: Location
plasecName: Name
plasecPartitionDN: Partitions
plasecTimezoneDN: Schedule
plasecoutputMode: Operating Mode
plasecoutputOfflinemode: Offline Mode
plasecoutputPulsetime: Pulse Time
plasecoutputTrigger: Trigger
```

Panels

Model: panels

cn: CN

description: Description

plasecAddress: Address

plasecAltname: Alt Name

plasecId: ID

plasecInstalled: Installed

plasecLocation: Location

plasecName: Name

plasecPartitionDN: Partitions

ipNetmaskNumber: Netmask

ipNetworkNumber: IP Address

macAddress: MAC Address

plasecAcenable: Power Fail Enable

plasecBatenable: Low Batt

plasecGateway: Appliance

plasecLocalTimeStr: Timezone

plasecTamperenable: Tamper Enable

plasecUltamperenable: UL Tamper Enable

plasecpanelAcceptport: Accept Port

plasecpanelAdditionalMemory: Additional Memory

plasecpanelAltportcomms: Alt Comms

plasecpanelAltportenable: Alternate comms

plasecpanelAltportipaddr: Alt IP address

plasecpanelAltportnetretry: Alt reply timeout

plasecpanelAltportreplyto: Alt poll delay

plasecpanelBaud: Baud

plasecpanelDownstreamenable: Enable Downstream Devices

plasecpanelEnablePIV: Enable PIV Cards

```
plasecpanelEncrypt: Use encryption
plasecpanelInput1enable: Input1 Enable
plasecpanelInput1masked: Input1 Masked
plasecpanelInput2enable: Input2 Enable
plasecpanelInput2masked: Input2 Masked
plasecpanelIsIPClient: IP Client Connection
plasecpanelListenport: Port
plasecpanelManuf: Vendor
plasecpanelMaxfloors: Max Elevator Floors
plasecpanelMkey1: M-Key1
plasecpanelMkey2: M-key2
plasecpanelModel: Model
plasecpanelNetretry: Retries
plasecpanelOfflineto: Offline timeout
plasecpanelPhysicallocation: Physical Location
plasecpanelPortwiringtype: Wiring Type
plasecpanelRelay1duration: Relay1 Duration
plasecpanelRelay1mode: Relay1 Mode
plasecpanelRelay2duration: Relay2 Duration
plasecpanelRelay2mode: Relay2 Mode
plasecpanelReplyto: Reply timeout
plasecpanelRequireTLS: TLS Required
plasecpanelRtscontrol: RTS Control
plasecpanelTimeoutdiscon: Poll Delay
```

Find Installed or Uninstalled Panels

Note: The following requests and responses are supported in API version 1.11 or later.

Request Examples

XML

```
https://<my_ACM_Server>/rest/panels.xml?attributes=plasecInstalled,entryUUID,
dn,plasecName,plasecPartitionDN&installed=true
```

JSON

```
https://<my_ACM_Server>/rest/panels.json?attributes=plasecInstalled,entryUUID,
dn,plasecName,plasecPartitionDN&installed=true
```

Show Panel Status

Request Example

Get the panel status.

> **Note:** Only the XML-based request is supported.

XML

```
GET /appliances/panels_status.xml
```

Response Example

```
RESPONSE: #<Net::HTTPOK:0x00000001570f2708>

DATA:
<?xml version="1.0" encoding="UTF-8"?>
<objects type="array">
  <object>
    <comm-status type="integer">0</comm-status>
    <manuf type="integer">1</manuf>
    <pmodel type="integer">8</pmodel>
    <pmodel-actual type="integer">0</pmodel-actual>
    <type-mismatch type="integer">0</type-mismatch>
    <id-mismatch type="integer">0</id-mismatch>
    <max-ch-mismatch type="integer">0</max-ch-mismatch>
    <firmware-download type="integer">0</firmware-download>
    <parameter-download type="integer">0</parameter-download>
    <token-download type="integer">0</token-download>
    <serial type="integer">1</serial>
    <dip type="integer">0</dip>
    <last-comms type="integer">0</last-comms>
```

```
<clock type="integer">0</clock>
<firmware-rev>0.0.0</firmware-rev>
<triggerCount type="integer">0</triggerCount>
<inp-cnt type="boolean">false</inp-cnt>
<out-cnt type="boolean">false</out-cnt>
<rdr-cnt type="boolean">false</rdr-cnt>
<mem-size type="integer">0</mem-size>
<mem-avail type="integer">0</mem-avail>
<cards-max type="integer">0</cards-max>
<cards-avail type="integer">0</cards-avail>
<db-version type="integer">0</db-version>
<alarm-count type="integer">0</alarm-count>
<inputs>
  <16 type="integer">0</16>
  <17 type="integer">0</17>
  <18 type="integer">0</18>
  <19 type="integer">0</19>
</inputs>
<doors type="array"/>
<subp>
  <0>
    <status type="integer">0</status>
    <type-mismatch type="integer">0</type-mismatch>
    <type type="integer">0</type>
    <tm-last-status type="integer">0</tm-last-status>
    <firmware-rev>0.0</firmware-rev>
    <firmware-download type="integer">0</firmware-download>
    <alarm-count type="integer">0</alarm-count>
    <inputs>
      <0 type="integer">32768</0>
      <1 type="integer">34816</1>
      <193 type="integer">0</193>
      <194 type="integer">0</194>
      <195 type="integer">0</195>
    </inputs>
    <inputs-alarm-count>
      <0 type="integer">0</0>
      <1 type="integer">0</1>
      <193 type="integer">0</193>
      <194 type="integer">0</194>
      <195 type="integer">0</195>
    </inputs-alarm-count>
    <outputs>
      <0 type="integer">32768</0>
      <1 type="integer">32768</1>
    </outputs>
    <outputs-alarm-count>
      <0 type="integer">0</0>
      <1 type="integer">0</1>
    </outputs-alarm-count>
```

```
      <doors>
        <1 type="integer">0</1>
        <2 type="integer">0</2>
      </doors>
    </0>
  </subp>
  <gateway-name>ACMvm101</gateway-name>
  <plasecInstalled type="boolean">false</plasecInstalled>
  <plasecpanelManuf>cn=1,ou=manufs,dc=plasec</plasecpanelManuf>
  <gateway-id>abcd1234</gateway-id>
  <plasecName>test panel one</plasecName>
  <panel-name>test panel one</panel-name>
  <panel-id>1</panel-id>
  <bat-enable nil="true"/>
</object>
<object>
  <comm-status type="integer">0</comm-status>
  <manuf type="integer">1</manuf>
  <pmodel type="integer">6</pmodel>
  <pmodel-actual type="integer">0</pmodel-actual>
  <type-mismatch type="integer">0</type-mismatch>
  <id-mismatch type="integer">0</id-mismatch>
  <max-ch-mismatch type="integer">0</max-ch-mismatch>
  <firmware-download type="integer">0</firmware-download>
  <parameter-download type="integer">0</parameter-download>
  <token-download type="integer">0</token-download>
  <serial type="integer">5</serial>
  <dip type="integer">0</dip>
  <last-comms type="integer">0</last-comms>
  <clock type="integer">0</clock>
  <firmware-rev>0.0.0</firmware-rev>
  <triggerCount type="integer">0</triggerCount>
  <inp-cnt type="boolean">false</inp-cnt>
  <out-cnt type="boolean">false</out-cnt>
  <rdr-cnt type="boolean">false</rdr-cnt>
  <mem-size type="integer">0</mem-size>
  <mem-avail type="integer">0</mem-avail>
  <cards-max type="integer">0</cards-max>
  <cards-avail type="integer">0</cards-avail>
  <db-version type="integer">0</db-version>
  <alarm-count type="integer">0</alarm-count>
  <inputs>
    <16 type="integer">0</16>
    <17 type="integer">0</17>
    <18 type="integer">0</18>
    <19 type="integer">0</19>
  </inputs>
  <doors type="array"/>
  <subp>
    <0>
```

```
        <status type="integer">0</status>
        <type-mismatch type="integer">0</type-mismatch>
        <type type="integer">0</type>
        <tm-last-status type="integer">0</tm-last-status>
        <firmware-rev>0.0</firmware-rev>
        <firmware-download type="integer">0</firmware-download>
        <alarm-count type="integer">0</alarm-count>
        <inputs>
          <0 type="integer">32768</0>
          <1 type="integer">32768</1>
          <193 type="integer">0</193>
          <194 type="integer">0</194>
          <195 type="integer">0</195>
        </inputs>
        <inputs-alarm-count>
          <0 type="integer">0</0>
          <1 type="integer">0</1>
          <193 type="integer">0</193>
          <194 type="integer">0</194>
          <195 type="integer">0</195>
        </inputs-alarm-count>
        <outputs>
          <0 type="integer">32768</0>
          <1 type="integer">32768</1>
        </outputs>
        <outputs-alarm-count>
          <0 type="integer">0</0>
          <1 type="integer">0</1>
        </outputs-alarm-count>
        <doors>
          <1 type="integer">0</1>
          <2 type="integer">0</2>
        </doors>
      </0>
    </subp>
    <gateway-name>ACMvm101</gateway-name>
    <plasecInstalled type="boolean">false</plasecInstalled>
    <plasecpanelManuf>cn=1,ou=manufs,dc=plasec</plasecpanelManuf>
    <gateway-id>abcd1234</gateway-id>
    <plasecName>test 1501</plasecName>
    <panel-name>test 1501</panel-name>
    <panel-id>5</panel-id>
    <bat-enable nil="true"/>
  </object>
</objects>
```

Photos

See *Identities* on page 104.

Readers

Model: readers

```
cn: CN
description: Description
plasecAddress: Address
plasecAltname: Alt Name
plasecId: ID
plasecInstalled: Installed
plasecLocation: Location
plasecName: Name
plasecPartitionDN: Partitions
plasecDoorDN: Door Name
plasecIdentityDN: Identity
plasecTimezoneDN: Schedule
plasecreaderBidirect: Bidirectional
plasecreaderC52Beep: Double Beep on Access Grant
plasecreaderC5Chargeonly: Card Charger Only
plasecreaderC5Wiegandport: Wiegand Port
plasecreaderC5baud: Baud Rate
plasecreaderC5hidprox: Use HID Prox
plasecreaderC5port: Port
plasecreaderF2FDecoding: F/2F Decoding
plasecreaderFmtnbl: Format by nibble
plasecreaderInputsonreader: Inputs on reader
plasecreaderKeypadDecoding: Keypad Decode
plasecreaderLeddrivemode: LED drive
plasecreaderNcimag: NCI magstripe
plasecreaderSupervised: Supervised
plasecreaderTrimzbit: Trim Zero Bit
plasecreaderTyp: Type
```

```
plasecreaderWeigand: Weigand
plasecreaderWeigandport: Weigand
plasecroleAccessgroupDN: Access Group
```

Roles

A role specifies all the permissions an ACM operator needs in the ACM system to perform a specific function in the organization. Each role can include access groups, delegations, routing groups, and role-assignment privileges.

Model: roles

```
cn: CN
description: Description
plasecAddress: Address
plasecAltname: Alt Name
plasecId: ID
plasecInstalled: Installed
plasecLocation: Location
plasecName: Name
plasecPartitionDN: Partitions
parent_dn: Parent
plasecActivatedate: Start Date
plasecDeactivatedate: Stop Date
plasecGrouptype: Group Type
plasecTimezoneDN: Schedule
plasecroleAccessgroupDN: Access Group Key
plasecroleActive: Active Flag
plasecroleDoorDN: Door Key
plasecroleIdentityDN: Identity Key
plasecroleParentDN: Parent
plasecrolePartitiongroupDN: Partition Group Key
plasecroleRightgroupDN: Delegations
plasecroleRoutinggroupDN: Routing
plasecroleSODrolesDN: SOD Roles Key
plasecroleStartdate: Start Date
plasecroleStopdate: Stop Date
```

```
plasecroleUdeftabDN: Udef Table
plasecrolerouteAck: Ack
plasecrolerouteClear: Clear
plasecrolerouteEvtypeDN: Evtype Key
plasecrolerouteGroupDN: Group Key
plasecrolerouteTimezoneDN: Schedule Key
```

Find All Roles

Note: The following requests and responses are supported in API version 1.0 or later.

Request and Response Examples

XML

Request:

```
https://<my_ACM_Server>/roles.xml
```

Response:

```
<?xml version="1.0" encoding="UTF-8"?>
<roles type="array">
  <role>
    <dn>cn=9,ou=roles,dc=plasec</dn>
    <cns type="array">
      <cn>9</cn>
    </cns>
    <createTimestamp>20200831054425Z</createTimestamp>
    <creatorsName>cn=manager,dc=plasec</creatorsName>
    <entryDN>cn=9,ou=roles,dc=plasec</entryDN>
    <entryUUID>c5c4d714-7f98-103a-8396-4baa5bf33c8a</entryUUID>
    <hasSubordinates>FALSE</hasSubordinates>
    <modifiersName>cn=manager,dc=plasec</modifiersName>
    <modifyTimestamp>20211213230115Z</modifyTimestamp>
    <objectClasses type="array">
      <objectClass>plasecRole</objectClass>
    </objectClasses>
    <plasecActivatedate>20081009000000Z</plasecActivatedate>
    <plasecDeactivatedate>30010101075959Z</plasecDeactivatedate>
    <plasecInstalled>TRUE</plasecInstalled>
    <plasecName>ACC Administrator</plasecName>
    <plasecroleActive>TRUE</plasecroleActive>
    <plasecroleDefaultRole>FALSE</plasecroleDefaultRole>
```

```
    <plasecroleNeverExpire>TRUE</plasecroleNeverExpire>
    <plasecroleRightgroupDNs type="array">
      <plasecroleRightgroupDN>cn=acm-
1,ou=rightgroups,ou=rights,dc=plasec</plasecroleRightgroupDN>
    </plasecroleRightgroupDNs>
    <structuralObjectClass>plasecRole</structuralObjectClass>
    <subschemaSubentry>cn=Subschema</subschemaSubentry>
  </role>
  <role>
    <dn>cn=7,ou=roles,dc=plasec</dn>
    <cns type="array">
      <cn>7</cn>
    </cns>
    <createTimestamp>20200831054424Z</createTimestamp>
    <creatorsName>cn=manager,dc=plasec</creatorsName>
    <entryDN>cn=7,ou=roles,dc=plasec</entryDN>
    <entryUUID>c4fd6530-7f98-103a-82f7-4baa5bf33c8a</entryUUID>
    <hasSubordinates>FALSE</hasSubordinates>
    <modifiersName>cn=manager,dc=plasec</modifiersName>
    <modifyTimestamp>20210817175427Z</modifyTimestamp>
    <objectClasses type="array">
      <objectClass>plasecRole</objectClass>
    </objectClasses>
    <plasecActivatedate>20081009000000Z</plasecActivatedate>
    <plasecDeactivatedate>20201231000000Z</plasecDeactivatedate>
    <plasecInstalled>TRUE</plasecInstalled>
    <plasecName>ACM Verify Administrator</plasecName>
    <plasecroleActive>FALSE</plasecroleActive>
    <plasecroleDefaultRole>FALSE</plasecroleDefaultRole>
    <plasecroleRightgroupDNs type="array">

<plasecroleRightgroupDN>cn=8,ou=rightgroups,ou=rights,dc=plasec</plasecroleRig
htgroupDN>
    </plasecroleRightgroupDNs>
    <structuralObjectClass>plasecRole</structuralObjectClass>
    <subschemaSubentry>cn=Subschema</subschemaSubentry>
  </role>
  …
</roles>
```

Get Installed or Uninstalled Roles

Note: The following requests and responses are supported in API version 1.11 or later.

Request Examples

Get all the installed roles by specifying role attributes.

XML

```
https://<my_ACM_Server>/rest/roles.xml?attributes=dn,plasecName,plasecrole
RightgroupDN,plasecInstalled,plasecActivatedate,plasecDeactivatedate,plasecPar
titionDN&installed=true
```

JSON

```
https://<my_ACM_Server>/rest/roles.json?attributes=dn,plasecName,plasecrole

RightgroupDN,plasecInstalled,plasecActivatedate,plasecDeactivatedate,plasecPar
titionDN&installed=true
```

Schedules

Model: timezones

```
cn: CN
description: Description
plasecAddress: Address
plasecAltname: Alt Name
plasecId: ID
plasecInstalled: Installed
plasecLocation: Location
plasecName: Name
plasecPartitionDN: Partitions
plasectzoneMode: Mode
```

Subpanels

If your system supports a hierarchical structure, subpanels are specialty panels that are controlled by a master panel.

One master panel can be connected to a large number of specialty subpanels, incorporating an array of devices such as single and multiple door modules, keypads, input modules and output modules.

Model: subps

```
cn: Common Name

description: Description

dn: Distinguished Name

entryUUID: Universally Unique Identifier of the entry

plasecAddress: Address

plasecAltname: Alt Name

plasecId: ID

plasecInstalled: Installed

plasecLocation: Location

plasecName: Name

plasecPartitionDN: Partitions

ipNetworkNumber: IP Address

macAddress: MAC Address

plasecAcenable: AC Enable

plasecBatenable: Bat Enable

plasecTamperenable: Tamper Enable

plasecUltamperenable: UL Tamper Enable

plasecpanelEncrypt: Encrypt

plasecsubpCinputs: C-inputs

plasecsubpCoutputs: C-outputs

plasecsubpCreaders: C-readers

plasecsubpCsubpin: Continue Inputs

plasecsubpCsubpout: Continue Outputs

plasecsubpCsubprdr: Continue Readers
```

```
plasecsubpElevinput: Elevator Inputs

plasecsubpElevoutput: Elevator Outputs

plasecsubpModel: Model

plasecsubpPhysicallocation: Physical Location

plasecsubpPortadds: Port

plasecsubpLowdr: Low Door

plasecsubpHighdr: High Door
```

Sample Responses

All Subpanels for a Single Panel

> **Note:** The following output is supported in ACM API version 1.0 or later.

The following response has retrieved all subpanels that belong to a panel specified by its CN identifier, and the available attributes of the subpanels.

```
<subps type="array">

<subp>

<dn>cn=0,cn=2,ou=panels,cn=f9250f84d98645e2,ou=gateways,dc=plasec</dn>

<cns type="array">

<cn>0</cn>

</cns>

<entryUUID>89726794-1c51-103b-84ba-a7d26d84c60d</entryUUID>

<objectClasses type="array">

<objectClass>plasecSubpanel</objectClass>

</objectClasses>

<plasecAcenable>TRUE</plasecAcenable>

<plasecAddress>0</plasecAddress>

<plasecBatenable>TRUE</plasecBatenable>

<plasecInstalled>TRUE</plasecInstalled>

<plasecName>LP 1501 downstream - 1501 Internal SIO-0</plasecName>

<plasecTamperenable>TRUE</plasecTamperenable>
```

<plasecUltamperenable>FALSE</plasecUltamperenable>

<plasecsubpCinputs>0</plasecsubpCinputs>

<plasecsubpCoutputs>0</plasecsubpCoutputs>

<plasecsubpCreaders>0</plasecsubpCreaders>

<plasecsubpModel>88</plasecsubpModel>

<plasecsubpPortadds>0</plasecsubpPortadds>

</subp>

<subp>

<dn>cn=1,cn=2,ou=panels,cn=f9250f84d98645e2,ou=gateways,dc=plasec</dn>

<cns type="array">

<cn>1</cn>

</cns>

<entryUUID>974a755a-1c51-103b-84c6-a7d26d84c60d</entryUUID>

<objectClasses type="array">

<objectClass>plasecSubpanel</objectClass>

</objectClasses>

<plasecAcenable>TRUE</plasecAcenable>

<plasecAddress>1</plasecAddress>

<plasecBatenable>TRUE</plasecBatenable>

<plasecInstalled>TRUE</plasecInstalled>

<plasecName>LP 1501 downstream-LP1501-with Downstream-1</plasecName>

<plasecTamperenable>TRUE</plasecTamperenable>

<plasecUltamperenable>FALSE</plasecUltamperenable>

<plasecsubpCinputs>0</plasecsubpCinputs>

<plasecsubpCoutputs>0</plasecsubpCoutputs>

<plasecsubpCreaders>0</plasecsubpCreaders>

<plasecsubpModel>143</plasecsubpModel>

<plasecsubpPortadds>1</plasecsubpPortadds>

</subp>

</subps>

All Subpanels for All Panels

Note: The following output is supported in ACM API version 1.05 or later.

The following response has retrieved all subpanels for all panels in ACM, and a subset of the attributes of each subpanel.

```
<subps type="array">
<subp>
<dn>cn=0,cn=2,ou=panels,cn=f9250f84d98645e2,ou=gateways,dc=plasec</dn>
<plasecName>LP 1501 downstream - 1501 Internal SIO-0</plasecName>
<plasecInstalled>TRUE</plasecInstalled>
<entryUUID>89726794-1c51-103b-84ba-a7d26d84c60d</entryUUID>
</subp>
<subp>
<dn>cn=1,cn=2,ou=panels,cn=f9250f84d98645e2,ou=gateways,dc=plasec</dn>
<plasecName>LP 1501 downstream-LP1501-with Downstream-1</plasecName>
<plasecInstalled>TRUE</plasecInstalled>
<entryUUID>974a755a-1c51-103b-84c6-a7d26d84c60d</entryUUID>
</subp>
<subp>
<dn>cn=0,cn=3,ou=panels,cn=f9250f84d98645e2,ou=gateways,dc=plasec</dn>
<plasecName>HID V1000 - V1000 Internal SIO-0</plasecName>
<plasecInstalled>TRUE</plasecInstalled>
<entryUUID>bf606594-1c75-103b-84ea-a7d26d84c60d</entryUUID>
</subp>
<subp>
<dn>cn=1,cn=3,ou=panels,cn=f9250f84d98645e2,ou=gateways,dc=plasec</dn>
<plasecName>HID V1000 - V100-01</plasecName>
<plasecInstalled>FALSE</plasecInstalled>
```

```
<entryUUID>cf6de5a6-1c75-103b-84f4-a7d26d84c60d</entryUUID>
</subp>
</subps>
```

Find Installed or Uninstalled Subpanels

Note: The following requests and responses are supported in API version 1.11 or later.

Request Examples

XML

```
https://<my_ACM_Server>/rest/subps.xml?attributes=plasecName,dn,isInUse,
plasecPartitionDN,entryUUID,plasecInstalled&installed=true
```

JSON

```
https://<my_ACM_Server>/rest/subps.json?attributes=plasecName,dn,isInUse,
plasecPartitionDN,entryUUID,plasecInstalled&installed=true
```

Show Subpanel Status

Request Example

Get the panel and subpanel status.

Note: Only the XML-based request is supported.

XML

```
GET /appliances/panels_status.xml
```

Response Example

For a response example, see *Response Example* on page 148 for panels.

Tokens

Token records contain the unique identifier that opens the doors. The attribute plasecInternalnumber is the actual token number from the card that is read by a card reader.

Tokens are under Identities in the database tree and so most operations on tokens require that you know the Identity. There are some special cases though for example you can query for a token number (plasecInternalnumber) directly.

Model: tokens

```
cn: CN
description: Description
plasecAddress: Address
plasecAltname: Alt Name
plasecId: ID
plasecInstalled: Installed
plasecLocation: Location
plasecName: Name
plasecPartitionDN: Partitions
plasecActivatedate: Activate Date
plasecAreaDN: Last Area
plasecApbloc: APB loc
plasecDeactivatedate: Deactivate Date
plasecDownload: Download
plasecEmbossednumber: Embossed Number
plasecInternalnumber: Internal Number
plasecIssuedate: Issue Date
plasecLastdoor: Last Door
plasecLasttime: Last Used
plasecPIN: PIN
plasecThreat: Threat
plasecTokenVIP: APB Exempt
plasecTokenlevel: Issue Level
```

```
    plasecTokennoexpire: Never Expire
    plasecTokenstatus: Token Status
    plasecTrace: Trace
    plasecUdefstatus: Status
    plasecUsecount: Use Count
    plasecgwCsc5listen: C5 Listen Port
    plasecgwCslogmissing: Log Missing After
    plasecidentityRoleDN: Role
    plasectokenExtaccess: Extended Door Times
    plasectokenIdentityroleDN: Identity Role Key
    plasectokenPinexempt: Pin exempt
    plasectokenUseloseexempt: Use/Lose exempt
```

Find All

```
# tokens_all.rb
# sample REST call to get a list of all tokens
# <twl> Avigilon, Inc.
#
# arguments: ServerAddress, ServerPort, Username, password
#
my_server = ARGV[0]
my_port = ARGV[1]
my_user = ARGV[2]
my_pwd = ARGV[3]

require 'net/https'

http = Net::HTTP.new(my_server, my_port)
http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE

http.start do |http|

  # this request is for a list of all tokens
  # NOTE: pass in the identity CN to get tokens for an identity.
  #       pass in "-1" to get all the TOKENS
  req = Net::HTTP::Get.new('/identities/-1/tokens.xml')

  # gotta give our login and password for authorization and authentication
  req.basic_auth(my_user, my_pwd)

  # set the content type to xml for REST commands
```

```
  req["Content-Type"] = "application/xml"

  #send the REST request
  resp = http.request(req)

  puts "RESPONSE: #{resp.header} \n\n"
  puts "DATA: "
  puts "#{resp.body}"
  puts "#{resp.header['location']}"

end
```

Find by Identity

```
# token_by_id.rb
# sample REST call to get the tokens given an identity CN
# <twl> Avigilon, Inc.
#
# arguments: ServerAddress, ServerPort, Username, password
#
my_server = ARGV[0]
my_port = ARGV[1]
my_user = ARGV[2]
my_pwd = ARGV[3]
my_cn = ARGV[4]

require 'net/https'

http = Net::HTTP.new(my_server, my_port)
http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE

http.start do |http|

  # this request is for a list of all tokens for a given identity CN
  # NOTE a -1 for my_cn would tell us to get all tokens
  req = Net::HTTP::Get.new("/identities/#{my_cn}/tokens.xml")

  # gotta give our login and password for authorization and authentication
  req.basic_auth(my_user, my_pwd)

  # set the content type to xml for REST commands
  req["Content-Type"] = "application/xml"

  #send the REST request
  resp = http.request(req)

  puts "RESPONSE: #{resp.header} \n\n"
  puts "DATA: "
```

```
  puts "#{resp.body}"

end
```

Token Create

```
# token_create.rb
# sample REST call to create a new token for an identity
# <twl> Avigilon, Inc.
#
# arguments: ServerAddress, ServerPort, Username, password
#
my_server = ARGV[0]
my_port = ARGV[1]
my_user = ARGV[2]
my_pwd = ARGV[3]
require 'net/https'

http = Net::HTTP.new(my_server, my_port)
http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE

http.start do |http|

  # this request is to create an token
  #
  # NOTE: the CN of the identity for which the token should be created is
between 'identities' and 'tokens.xml'
  #
  #
  # required:
  #  - plasecInternalnumber is required, must be an integer and must be unique
within the system
  #  - plasecTokenstatus is required, (1==active, anything else is inactive
based on the user defined token status lookup)
  #      token can only be marked active if the identity is active, token can
only be marked active if the
  #      Deactivedate has not been reached and the Activedate is in the past
  #  - cannot exceed the max_active_tokens, plasecidentityMaxActiveTokens or
the system setting Max Active Tokens.  plasecidentityMaxActiveTokens takes
precedence
  #  - If given the plasecPIN must be numeric, cannot exceed 8 characters in
length and cannot be used by another identity
  #  - plasecIssuedate is required
  #  - plasecActivatedate is required
  #
  #
  ## NOTE: plasecIssuedate is required and passed on the url
  ##       plasecActivatedate is required and  passed on the url
```

```
  ##      plasecDeactivatedate is a param on the url (set to 12/31/2037 if not
passed in)
  ## SPECIAL Case that these need to be passed in on the url - due to the way
rails parses the xml
  #
  # The Identity CN is passed in the URL to identity which identity we are
creating this token for, of course this must be a valid identity

  req = Net::HTTP::Post.new
('/identities//5bc2ab7de9c843d9/tokens.xml?plasecIssuedate=01/10/2015&plasecAc
tivatedate=01/10/2015&plasecDeactivatedate=12/31/2037')

  # The body of the request contains xml of the attribute(s) to update
  # NOTE: To reference a PIV card, specify plasecTokenType 2 and to reference
a PIV-I card, specify plasecTokenType 3
```

Note: `plasecTokenType` is supported in ACM API version 1.02 or later.

```
  xml_string =
  '<token>
    <plasecTokenType>2</plasecTokenType>
    <plasecDownload>TRUE</plasecDownload>
    <plasecTokenstatus>1</plasecTokenstatus>
    <plasecTrace>TRUE</plasecTrace>
    <plasecTokenlevel>6</plasecTokenlevel>
    <plasecTokennoexpire>TRUE</plasecTokennoexpire>
    <plasectokenExtaccess>TRUE</plasectokenExtaccess>
    <plasecTokenVIP>TRUE</plasecTokenVIP>
    <plasecEmbossednumber>999111</plasecEmbossednumber>
    <plasecPIN>999911</plasecPIN>
    <plasecInternalnumber>999911</plasecInternalnumber>
    </token>'

  req.body = xml_string

  # gotta give our login and password for authorization and authentication
  req.basic_auth(my_user, my_pwd)

  # set the content type to xml for REST commands
  req["Content-Type"] = "application/xml"

  #send the REST request
  resp = http.request(req)

  puts "RESPONSE: #{resp.header} \n\n"
  puts "DATA: "
  puts "#{resp.body}"
```

```
  puts "#{resp.header['location']}"

end
```

Token Delete

```
# token_delete.rb
# sample REST call to delete a new token for an identity
# <twl> Avigilon, Inc.
#
# arguments: ServerAddress, ServerPort, Username, password
#
my_server = ARGV[0]
my_port = ARGV[1]
my_user = ARGV[2]
my_pwd = ARGV[3]
require 'net/https'

http = Net::HTTP.new(my_server, my_port)
http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE

http.start do |http|


  # this request is to delete one specific token for this identity
  # The url passed in contains the identity CN and the Token CN
  # The HTTP type of DELETE is used to delete the record
  #

  req = Net::HTTP::Delete.new('/identities/5bc2ab7de9c843d9/tokens/1.xml')

  # gotta give our login and password for authorization and authentication
  req.basic_auth(my_user, my_pwd)

  # set the content type to xml for REST commands
  req["Content-Type"] = "application/xml"

  #send the REST request
  resp = http.request(req)

  puts "RESPONSE: #{resp.header} \n\n"
  puts "DATA: "
  puts "#{resp.body}"
  puts "#{resp.header['location']}"

end
```

Token Update

```
# token_update.rb
# sample REST call to update a token
# <twl> Avigilon, Inc.
#
# arguments: ServerAddress, ServerPort, Username, password
#
my_server = ARGV[0]
my_port = ARGV[1]
my_user = ARGV[2]
my_pwd = ARGV[3]
require 'net/https'

http = Net::HTTP.new(my_server, my_port)
http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE

http.start do |http|

  # this request is to update an token record
  # the CN of the identity for which the token should be created is between
'identities' and 'tokens/.xml'
  ## NOTE: plasecIssuedate is required and  passed on the url
  ##       plasecActivatedate is required and  passed on the url
  ##      plasecDeactivatedate is a param on the url (set to 12/31/2037 if not
passed in)
  ## SPECIAL Case that these need to be passed in on the url - due to the way
rails parses the xml
  #
  # the identity CN and token CN are part of the url

  req = Net::HTTP::Put.new
('/identities/c756b34c61b842b6/tokens/3.xml?plasecDeactivatedate=12/31/2016&pl
asecIssuedate=01/01/2011&plasecActivatedate=01/01/2011')

  # The body of the request contains xml of the attribute(s) to update
  # attributes include the following
#    <plasecDownload>FALSE</plasecDownload>
#    <plasecTokenstatus>2</plasecTokenstatus>
#    <plasecTrace>FALSE</plasecTrace>
#    <plasecTokenlevel>9</plasecTokenlevel>
#    <plasecTokennoexpire>FALSE</plasecTokennoexpire>
#    <plasectokenExtaccess>TRUE</plasectokenExtaccess>
#    <plasecTokenVIP>TRUE</plasecTokenVIP>
#    <plasecEmbossednumber>111122222333334444</plasecEmbossednumber>
#    <plasecPIN>2222222</plasecPIN>
#    <plasecInternalnumber>1231231235555</plasecInternalnumber>
```

```
# NOTE: To reference a PIV card, specify plasecTokenType 2 and to reference  a
PIV-I card, specify plasecTokenType 3
```

Note: `plasecTokenType` is supported in ACM API version 1.02 or later.

```
 # This sample just updates the Token Status
 ##  Tokenstatus 1=Active, 2=Inactive
 xml_string =
 '<token>
  <plasecTokenstatus>2</plasecTokenstatus>
 </token>'

 req.body = xml_string

 # gotta give our login and password for authorization and authentication
 req.basic_auth(my_user, my_pwd)

 # set the content type to xml for REST commands
 req["Content-Type"] = "application/xml"

 #send the REST request
 resp = http.request(req)

 puts "RESPONSE: #{resp.header} \n\n"
 puts "DATA: "
 puts "#{resp.body}"

end
```

User Commands

Model: ucmds

cn: CN

description: Description

plasecAddress: Address

plasecAltname: Alt Name

plasecId: ID

plasecInstalled: Installed

plasecLocation: Location

plasecName: Name

plasecPartitionDN: Partitions

plasecucmdAccessdelay: Access Delay

plasecucmdAcclev: Acc Lev

plasecucmdArg1: Arg 1

plasecucmdArg2: Arg 2

plasecucmdArg3: Arg 3

plasecucmdArg4: Arg 4

plasecucmdAuthority: Authority

plasecucmdCmdcod: Cmd Cod

plasecucmdDoorarg: Door Arg

plasecucmdIdmode: Id Mode

plasecucmdMacroDN: Macro

plasecucmdTyp: Type

plasecucmdUsercmdtyp: User Cmd Type

plasecucmdUserlev: User Lev

plasecucmdValiddoors: Valid Doors

User Defined Tabs

Model: udef_tabs

```
cn: CN
description: Description
plasecAddress: Address
plasecAltname: Alt Name
plasecId: ID
plasecInstalled: Installed
plasecLocation: Location
plasecName: Name
plasecPartitionDN: Partitions
plasecudefTabfield: Tab Field
```

User Defined Lists

User-defined lists and their values are used for identity management in the ACM system.

Model: udef_vals

```
cn: CN
description: Description
dn: Distinguished Name
entryUUID: Universally Unique Identifier of the entry
plasecAddress: Address
plasecAltname: Alt Name
plasecId: ID
plasecInstalled: Installed
plasecLocation: Location
plaseclookupInuse: Lookup table entry is used or not used
plaseclookupValuetostore: Value used after lookup in Lookup table
plasecName: Name
plasecPartitionDN: Partitions
```

Output

Get All Values in a List

> **Note:** The following output is supported in ACM API version 1.06 or later.

The following output has retrieved the user-defined values specified by the CN identifier.

```
<lookups type="array">
<lookup>
<dn>cn=Warehouse,ou=lookups,dc=plasec</dn>
<cns type="array">
<cn>Warehouse</cn>
</cns>
<descriptions type="array">
```

```
<description>Warehouse</description>
</descriptions>
<entryUUID>40a94efa-3c55-103b-8de6-5fa0697c38cb</entryUUID>
<objectClasses type="array">
<objectClass>plasecLookup</objectClass>
</objectClasses>
<plaseclookupInuse>TRUE</plaseclookupInuse>
</lookup>
<lookup>
<dn>cn=Main Office,ou=lookups,dc=plasec</dn>
<cns type="array">
<cn>Main Office</cn>
</cns>
<descriptions type="array">
<description>Main Office</description>
</descriptions>
<entryUUID>40a94dd8-3c55-103b-8de5-5fa0697c38cb</entryUUID>
<objectClasses type="array">
<objectClass>plasecLookup</objectClass>
</objectClasses>
<plaseclookupInuse>TRUE</plaseclookupInuse>
</lookup>
</lookups>
```

User Defined Fields

Model: userdefs

```
cn: CN
description: Description
plasecAddress: Address
plasecAltname: Alt Name
plasecId: ID
plasecInstalled: Installed
plasecLocation: Location
plasecName: Name
plasecPartitionDN: Partitions
plasecUdef4: Udef 4
plasecUdef5: Udef 5
```